文档名称	企业服务总线系统建设项目服务接口技术规范	密级	秘密
文档版本	V1.3	页数	共 58 页

企业服务总线系统建设项目
服务接口技术规范

项目组：企业服务总线系统项目组

拟制	陈宏鸿、余以志、胡熠	日期	2013-06-19
评审人		日期	
批准人		日期	
签发		日期	

本文内容涉及浦发银行和神州数码融信软件有限公司商业秘密，未经书面许可，不得以任何形式披露、传播或扩散。©神州数码融信软件有限公司所有，保留所有权利

修订记录

编号	版本	修订内容	修订日期	修订人	审核
1.	V1.0	建立文档	2013/6/19	陈宏鸿	
2.	V1.1	增加内容说明，通讯规范，修订报文规范的内容。	2013/7/2	陈宏鸿	
3.	V1.2	取消 wsa 等命名空间，采用 ESB 统一命名空间 进一步明确报文命名空间的定义 修改流水号控制准则 增加 SOAP 的 WSDL 和样例	2013/7/8	陈宏鸿	
4.	V1.3	完善 SOAP 的报文样例	2013/7/9	蒋培，陈宏鸿	
5.					
6.					
7.					
8.					
9.					
10.					
11.					
12.					
13.					
14.					
15.					
16.					
17.					
18.					
19.					
20.					
21.					
22.					
23.					

目 录

1 概述.....5
1.1 概述说明.....5
1.2 内容说明.....5
1.3 编写目的.....5
1.4 目标读者.....5
1.5 术语说明.....6
2 ESB 系统定位.....9
2.1 针对银行内部的“交易线”应用集成.....9
2.2 功能定位于“服务集成”而不是“服务实现”.....9
3 ESB 通讯规范.....10
3.1 简单响应模式.....10
3.2 发布订阅模式.....11
3.3 文件传输模式.....12
4 ESB 报文规范.....13
4.1 SOAP 报文规范.....13
4.1.1 SOAP 头的信息(Header).....17
4.1.2 BODY 应用头信息（SHEAD）.....21
4.1.3 BODY 服务报文体（SBODY）.....23
4.1.4 系统异常返回（Fault）.....24
4.1.5 通讯方式.....25
4.1.6 报文编码.....25
4.1.7 报文样例.....25
4.2 SOP 报文规范.....35
4.2.1 概述.....35
4.2.2 单元定义.....36
4.2.3 输入信息格式.....38
4.2.4 输出信息格式.....41
4.2.5 打印模式定义.....42
4.2.6 通讯方式.....43
4.2.7 报文编码.....44
4.2.8 报文样例.....44
5 系统集成方法.....45
5.1 系统集成准则.....45
5.2 系统编码准则.....46
5.3 系统安全控制准则.....46
5.3.1 IP 地址控制.....46
5.3.2 服务权限控制.....46
5.3.3 服务调用时间控制.....46
5.3.4 报文完整性控制.....47
5.3.5 PIN 字段控制准则.....48
5.4 服务流量控制准则.....48

5.5 组合服务控制准则.....49
5.6 报文大小限制准则.....49
5.7 交易超时控制准则.....49
5.8 报文分页控制准则.....49
5.9 消息广播控制准则.....50
5.10 文件传输控制准则.....50
5.11 流水号控制准则.....51
5.12 服务 SLA 控制准则.....54
5.13 ESB 错误码列表.....54
6 应用举例.....57
6.1 服务消费者服务调用.....57
6.2 服务提供者服务发布.....57
7 参考附件.....58
7.1 浦发银行信息系统系统编号.....58
7.2 浦发银行 SOP 通讯报文格式.....58

1 概述

1.1 概述说明

本文档是浦发银行企业服务总线系统（简称 ESB）的技术规范。ESB 作为企业服务总线对外提供服务，服务的消费者需要调用 ESB 系统上的服务，服务的提供者需要在 ESB 系统上发布服务，相关的系统都需要遵循 ESB 提出来的技术规范，本文档旨在指导相关系统接入 ESB。

1.2 内容说明

该文档包含的内容主要涉及到的内容有如下几个方面：

1、 ESB 系统的功能定位。

2、 接入 ESB 系统的通讯协议规范。

3、 接入 ESB 系统的报文规范。

4、 外围系统与 ESB 系统间的技术集成原则，包括系统集成准则、系统编码准则、安全控制准则等一系列的内容。

5、 外围系统与 ESB 系统集成的相关举例。

1.3 编写目的

编写该文档的主要目标有下面几个方面：

1. 确定 ESB 系统所使用的报文标准、通讯协议和报文编码等技术方面的标准。
2. 确定 ESB 系统在集成服务消费者和服务提供者系统的架构准则。
3. 帮助 ESB 系统的服务消费者系统调用 ESB 系统上的服务。
4. 帮助 ESB 系统的服务提供者系统通过 ESB 系统对外发布服务。

1.4 目标读者

本文档是浦发银行新一代信息系统企业服务总线的技术规范文档，涉及到浦发银行 ESB 系统建设过程与外围系统相关的技术标准和系统约定。所以本文档的目标读者主要包括：

1. 浦发银行 ESB 系统项目组的系统架构师、系统分析师、服务规范人员和系统开发人员。
2. 浦发银行系统架构组的成员和新一代信息办公室的相关领导。
3. 浦发银行调用 ESB 系统所发布服务的相关系统的系统架构师和系统分析师。
4. 浦发银行通过 ESB 系统发布服务相关系统的系统架构师和系统分析师。

1.5 术语说明

序号	术语	术语说明
1.	ESB	企业服务总线系统，默认指“浦发银行企业服务总线系统”
2.	服务	ESB 系统上统一发布的服务接口，ESB 系统对外提供的接口。
3.	服务消费方	调用 ESB 系统上服务的系统称为“服务消费者”，也称为“Consumer”。
4.	服务提供方	通过 ESB 系统发布服务的系统称为为“服务提供者”，也称为“Provider”。
5.	标准接入	服务消费者按照浦发银行制定的标准报文和通讯方式调用服务的系统集成方式称为“标准接入“
6.	标准接出	服务提供者按照浦发银行制定的标准报文和通讯方式发布服务的系统集成方式称为“标准接出”。
7.	适配接入	除了“标准接入”以外其他方式的系统接入统称为“适配接入”。
8.	适配接出	除了“标准接出”以为其他方式的系统接出统称为“适配接出”。
9.	XML	“Extensible Markup Language”的简称，可扩展的标记语言。广泛应用与系统间的信息交互。
10.	Web Service	Web service 是一个平台独立的，松耦合的，自包含的、基于可编程的 web 的应用程序，可使用开放的 XML 标准来描述、发布、发现、协调和配置这些应用程序，用于开发分布式的互操作的应用程序。
11.	WS	“Web Service”的简称。
12.	UDDI	UDDI 是一套基于 Web 的、分布式的、为 Web Service 提供的、信息注册中心的实现标准规范，同时也包含一组使企业能将自身提供的 Web Service 注册，以使别的企业能够发现的访问协议的实现标准。(去除)
13.	SOAP	“Simple Object Access Protocol”的简称，简单对象访问协议。。SOAP 可以运行在任何其他传输协议上，在传输层之间的头是不同的，但 XML 有效负载保持相同。
14.	WSDL	Web Service 描述语言 WSDL 就是用机器能阅读的方式提供的一

		个正式描述文档而基于 XML 的语言，用于描述 Web Service 及其函数、参数和返回值。因为是基于 XML 的，所以 WSDL 是人与机器均可读取。
15.	Socket	通常称作"套接字"，用于描述 IP 地址和端口，是一个通信链的句柄。网络通讯的一套 API 实现 TCP 和 UDP 的网络通讯。Socket 通讯经常也被称为 TCP 通讯。
16.	SSL	“Secure Sockets Layer”的简称，带安全的套接层，既带安全的 Socket 通讯协议。
17.	HTTP	“Hypertext Transfer Protocol”的简称，超文本传输协议。HTTP 底层是基于 TCP/IP 协议。
18.	HTTPS	“Secure Hypertext Transfer Protocol”的简称，安全的超文本传输协议。
19.	JMS	“Java Message Service”的简称，Java 消息服务应用程序接口是一个 Java 平台中关于面向消息中间件。
20.	SOP	浦发银行核心系统所采用的报文格式称为 SOP，是联想核心系统所采用的报文格式。
21.	加密平台	浦发银行的内部系统，主要进行全行的信息加密、解密、转加密和密钥存储。
22.	数据架构组	浦发银行的内部组织结构。
23.	应用架构组	浦发银行的内部组织结构。
24.	信息标准组	浦发银行的内部组织结构。
25.	发布订阅	发布/订阅（Publish/subscribe 或 pub/sub）是一种消息范式，消息的发送者（发布者）不是计划发送其消息给特定的接收者（订阅者）。而是发布的消息分为不同的类别，而不需要知道什么样的订阅者订阅。
26.	文件传输	通过文件服务器进行文件的上传和下载。
27.	MD5	md5 是一个不可逆的单向散列算法。MD5 的应用主要用于校验内容的完整性。例如发送方根据算法生成报文摘要，并把报文摘要和报文一起发送给接收方，接收也根据报文和算法生成新的摘要，比较两个接收到的摘要和生成的摘要是否一致以此来达到校验完整性的过程。也广泛应用于数字签名。
28.	DES	DES（数据加密算法）是对称密码算法的一种，在数据安全方面有非常广泛的应用。 对称密码最大的特点是加密密码和解密密码相同。 DES 算法属于分组密码算法，以 64 位为一个分组，密码的实际长度也是 64 位，但是实际有效位数为 56 位。
29.	3DES	进行 3 次的 DES 加密运算，提供最长 168 位的密钥长度的加密算法。
30.	服务规范	ESB 系统所制定的服务规范。

31.	业务服务	ESB 系统对外发布供服务消费者系统调用的服务称为“业务服务”。
32.	基础服务	ESB 系统内部所使用的功能性的服务组件称为“集成服务”。
33.	组合服务	ESB 的流程引擎使用业务服务和基础服务组合起来形成功能更大的服务称为“组合服务”。
34.	SDO	“Service Data Objects”的简称，服务数据对象。
35.	SCA	“Service Component Architecture”的简称，即服务组件框架。
36.	IPP	浦发银行内部系统“综合前置”的简称。
37.	BIP	浦发银行内部系统“业务集成平台”的简称。
38.	浦发银行	此文档范围是“上海浦东发展银行”的简称。
39.	SPDB	“上海浦东发展银行”的英文简称。
40.	SPDBID	“浦发银行 ESB 系统技术规范”的简称。
41.	SPDBSD	“浦发银行 ESB 系统服务规范”的简称。
42.	神州数码	“神州数码融信软件有限公司”的简称。
43.	DCFS	“神州数码融信软件有限公司”的英文简称。

2 ESB 系统定位

企业服务总线系统建设的关键目标：是建立面向服务的 IT 总体架构，是 IT 系统适应快速发展、复杂多变的业务需求的最佳途径，是银行 IT 系统建设的趋势和方向，是实现银行 SOA 总体架构目标的关键和基础。

作为浦发银行应用的交易服务干线，企业服务总线系统在定位方面主要有以下几点：

2.1 针对银行内部的联机交易应用集成

银行的企业应用集成包括几个层面：

从集成的特点来看可以分为实时性要求高的联机交易服务整合（称之为“交易线应用整合”）和数据大批量处理模式的数据整合（称之为“数据线整合”）。

从应用集成的范围来看可以分为：银行内部应用整合，银行外部的 B2B 应用整合。

由于企业服务总线系统作为银行 IT 架构的核心枢纽主要负责银行内部的交易线的应用需求，企业服务总线系统关注的重点是浦发银行内部的交易线的应用整合。

ESB 是全行服务的统一注册、发布和管理的载体，统一的应用集成平台，用于实现行内系统间的互联互通，支持系统间服务的共享，推进系统间交互的标准化，屏蔽系统间接口的多样性，降低系统间的紧耦合关系，使服务资源得到共享和充分利用。

2.2 功能定位于“服务集成”而不是“服务实现”

企业服务总线是银行的基础设施系统，它的作用是为服务集成、服务和产品创新等应用提供基础架构的支撑，而不是去实现上述的集成业务应用。不支持业务处理逻辑，业务处理逻辑由各产品系统承载。

“交易线”应用整合的涉及到包括渠道整合、客户服务整合、服务流程整合等多个方面，这些整合应用包含复杂的业务规则和业务数据，这些业务逻辑不应由企业服务总线系统实现，而是由相应的应用系统负责具体实现，企业服务总线将这些应用集成起来，通过服务标准封装对外发布。ESB 系统关注的是如何可靠高效的保障行内实时交易系统的正常运行，实现敏捷快速的不受限制的系统集成，同时兼顾行内系统架构的现状和未来实现全行 SOA 架构的系统目标。

企业服务总线平台的功能包括：服务定义与发布，服务代理和服务路由，通讯连接和报文转换，服务资源管理，服务运行状态监控，服务水平协议管理 SLA。

3 ESB 通讯规范

ESB 支持的通讯消息交互模式主要包括有如下几种：

1) 简单请求响应模式（包括 one-way 模式）：一次请求对应一次响应或者没有响应。请求的处理可以是同步调用方式也可以是异步调用方式。
2) 一次请求对应多个响应模式：该模式主要用于处理大数据量交互。
3) 文件传输模式：一次请求后，响应信息被以文件的方式存放在制定的地方，由请求方自己获取，这种方式主要用于处理超大数据交互。处理的过程可以是同步调用方式，也可以是异步调用方式。
4) 订阅发布模式：发布订阅一般用于消息的广播和批量推送，推送过程较多是采用批量异步处理方式。

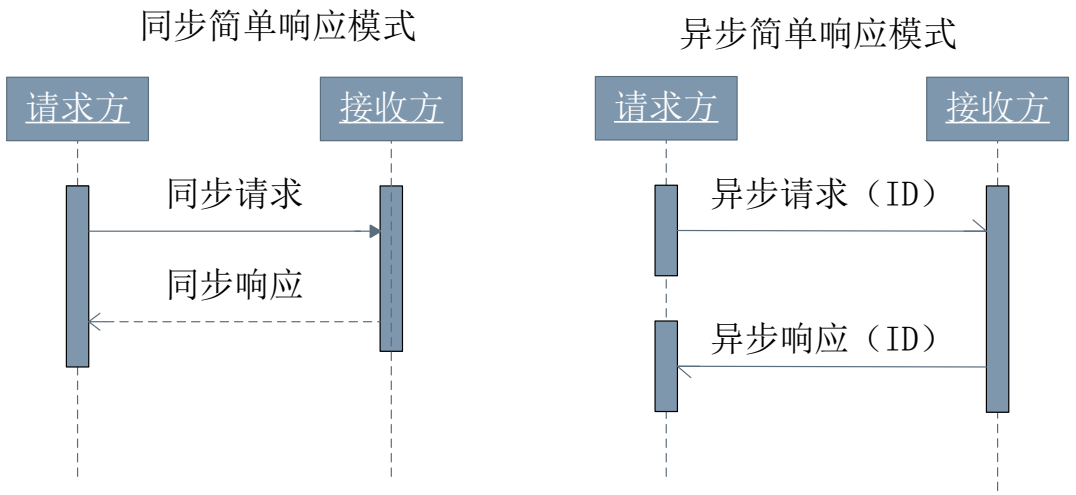
对于一次请求对应多种响应的模式，一般都是通过报文的分页查询来完成，或是采用文件传输的模式来完成大批量数据的交互，所以此模式没有很多的应用场景。

下面针对简单请求响应模式、文件传输模式和发布订阅模式的应用场景进行详细的说明。

3.1 简单响应模式

简单响应模式是银行业务最常用的消息交互模式，表现为一次请求对应一次响应或无响应。鉴于银行业务系统的特点需要确保消息的可靠传输，对于服务的请求需要给出明确的应答，所以虽然支持没有响应的请求，但是一般还是需要系统给出收到消息的应答。就像邮件的发送一样，邮件虽然是发送出去了，但是没有收到邮件回复或是邮件回执，是无法确保消息已经发送给接收的用户。所以无响应的模式虽然技术上是可行的，但是为了避免消息发送的不确定性，还是要求至少响应处理的状态成功或是失败。

简单响应模式支持同步调用和异步调用的机制。同步调用一般是在一个网络连接里面完成，通过 TCP 短连接和 HTTP 都能简单实现系统间的同步调用机制，并且业界通用的负载均衡设备对此两种协议的支持也非常好，实现系统的集群部署、系统横向扩展能力，提升系统的可靠性和安全性方面能起到非常好的效果。所以 ESB 系统将会支持 TCP 短连接和 HTTP 短连接的系统接入。对于后台系统的服务调用，则会按照原有系统的调用模式进行处理，以尽可能减少后台系统的改动为原则进行处理。



简单响应模式下的异步调用机制也可以通过 TCP 协议和 HTTP 协议来完成，与同步调用的区别主要有如下几个方面：

1. 调用方可以不需要同步等待消息的响应，可以释放处理的进程或是线程处理其他业务请求，减少系统资源的开销，提升系统的吞吐量。

2. 请求的消息和响应的消息可以不在同一个网络连接中完成，可能通过两个连接来共同完成，需要通过全局唯一的消息 ID 来匹配请求消息和响应消息的对应关系。控制复杂度较高，控制过程容易出现消息不匹配的错误。

3. 银行业务特点大部分都是人机交互的模式，所以需要增加同步转异步控制点，此过程可能在某一个渠道系统，也可能通过 ESB 系统完成。并需要增加超时的控制机制。

ESB 系统间的标准调用方式将会采用 WebService 进行服务的发布和调用，此种方式最通用的实现是通过 SOAP+HTTP 的方式来实现，所以主要是通过同步调用机制来实现服务的请求和调用。

当然 WebService 也可通过 JMS+HTTP 异步调用的机制来实现，无法通过负载均衡设备实现负载均衡，需要通过软件的方式实现负载均衡，所以在现阶段仅作为一个可能实现存在而不进行大范围的推广，在后续项目的建设过程中有可能采用。

3.2 发布订阅模式

在许多发布/订阅系统中，发布者发布消息到一个中间的消息代理，然后订阅者向该消息代理注册订阅，由消息代理来进行过滤。消息代理通常执行存储转发的功能将消息从发布者发送到订阅者。

所以 ESB 系统会充当这个消息代理的角色，并采用存储转发和服务调用方式来实现消息发布订阅的功能，实现一个消息向多个系统进行广播。在发布/订阅模型中有基于主题的和基于内容两种模式，在 ESB 的存储转发模块中都可以进行支持。

3.3 文件传输模式

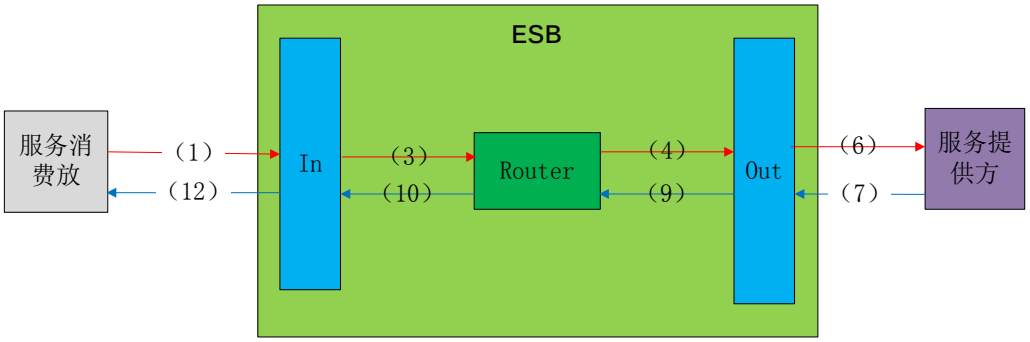
对于大批量的信息交互，简单通过请求响应的模式会遇到比较大的问题，主要的问题有如下两个：

1、 效率比较低下，通过 XML 的方式进行信息的交互，将会增加非常大的信息冗余，处理过程中大量的数据转换会极大降低系统的处理速度。

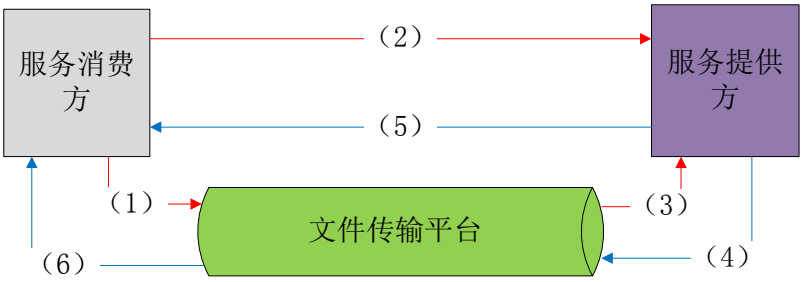
2、 大报文的处理和控制，对于系统的设计带来极大的挑战，对于短时间内集中的大报文访问容易造成系统资源的过度使用从而影响其他交易的正常处理。

所以为了支持大批量数据的应用场景，ESB 系统支持文件传输的路由控制，实现文件传输模式有如下几种：

1、 通过报文小附件的方式进行传输，既将文件内容通过压缩和 BASE64 编码的方式存放到报文中，作为报文中的小附件进行传输。适用于需要文件交互并且文件大小不会很大的情况下采用，整体报文大小受限于 ESB 系统对大报文的控制，例如报文最大不能超过 64K。

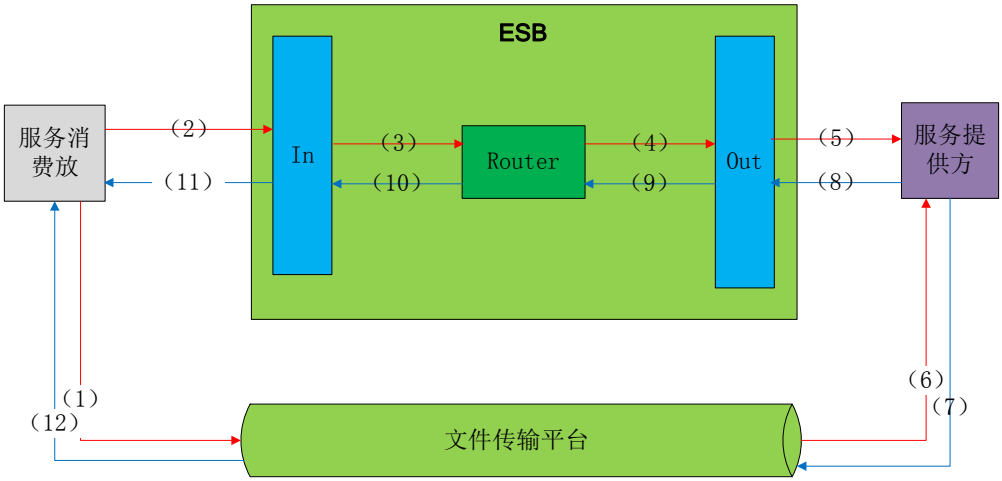


2、 点对点文件传输模式。既请求方和提供方按照系统约定进行文件传输，例如批量数据同步，可以在某一个约定时点完成。ESB 实现文件的路由。



3、 实时同步文件传输模式。请求方上传文件到对应的系统或是 ESB 文件服务器，并通过服务的方式通知提供方系统进行文件的处理；提供方提取文件后进行处理，处理完成后响应给请求方业务处理完成，处理状态是成功还是失败。请求方根据业务的要求确定是否需要

提取回盘文件。适用于处理时间不是很长的批量业务处理。



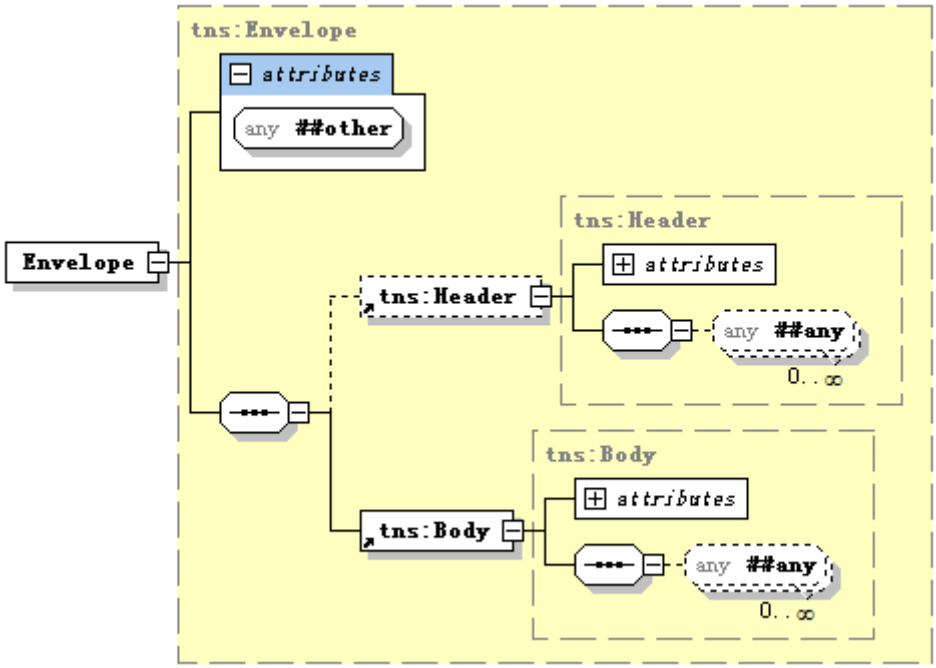
4、 实时异步文件传输模式。请求方上传文件到对应的系统或是 ESB 文件服务器，并通过服务的方式通知提供方系统进行文件的处理；提供方确定收到消息后响应给请求方消息已经收到，请求处理结束。提供方提取文件后进行处理，处理完成后发送另一个请求给请求方通知其处理的结果，如果有回盘文件会上传到对应的系统或是文件服务器上进行相关的处理。适用于大批量的业务数据处理，例如批量日终对账，批量签约等。

4 ESB 报文规范

为了更好的实现服务总线的建设目标，实现全行 SOA 架构的系统建设目标，需要确定浦发银行 ESB 建设的技术规范，其中很重要的一部分内容就是标准的报文规范。

4.1 SOAP 报文规范

SOAP 报文的结构如下图所示：



可以看到 SOAP 是一个信封头，里面包含信封头 Header，同时包含消息内容 Body，Header 和 Body 可由用户自行定义。

为了确定浦发银行 SOAP 报文的格式，兼顾报文的工业化标准，同时简化报文在处理过程中复杂的转换，并且尽可能短的压缩报文的长度，提升 ESB 系统在报文转换过程中的效率，现确定报文的规则如下：

(一)报文的规范采用 SOAP1.1 的标准进行信息的交互。

前缀	命名空间	描述
soap	http://www.w3.org/2003/05/soap-envelope	SOAP1.2（备选）
soap	http://schemas.xmlsoap.org/soap/envelope/	SOAP1.1
wsdl	http://schemas.xmlsoap.org/wsdl/	WSDL 的 schema
x	http://www.w3.org/2001/XMLSchema	Schema 定义
d	http://www.spdb.com.cn/spdb/metadata	浦发银行数据字典，默认命名空间
s	http://www.spdb.com.cn/spdb/service/[serviceid]	浦发银行服务结构定义命名空间，按服务进行区分

SOAP 最新版本是 SOAP 1.2，该版本提供了一个更严密、更可靠的规范集合，是基于协议和 XML 序列化方案进行绑定的一个抽象模型，更好地支持 WS 协议和 WS-Security 规范，功能扩展性上来说要好一些。但是目前各厂商对 SOAP1.1 的支持要到位一些，所以还是建议采用 SOAP1.1 的协议。

(二)通过 WSDL1.1 进行服务接口的发布。

WSDL（Web Service Definition Language）提供基于 XML 并且人机均可阅读的正式 Web Service 的描述文档，用于描述 Web Service 的方法、输入输出参数、协议和地址等信息。通过 WSDL 一些开发工具能够自动生成 Web service 的客户端代码和服务端代码。

所以 ESB 上服务接口的发布除了通过文档的形式对外发布外，还提供 WSDL 形式的服务发布功能。

(三)SOAP 的 XML 报文中引入命名空间。

引入 XML 的命名空间最基本的作用是通过前缀来避免命名冲突，但在 SOAP 报文中的应用主要有如下几个方面的作用：

1. SOAP 报文规范中使用 XML 的命名空间，至少也需要引入 SOAP 的命名空间和 XML Schema 的命名空间。
2. 通过命名空间，对服务接口数据定义中要用到的公共数据字段（数据字典）和公共结构体（如头信息）可存放到独立的 XSD 文件中，便于服务共享此部分的数据。避免公共数据修改对 WSDL 产生的影响。
3. 开源框架在生成 SOAP 报文的过程中，会自动引入命名的空间，不使用命名空间，对框架支持起来可能会遇到问题。

当然命名空间的使用也会带来一些问题：

1. 命名空间的引入会使得节点的长度变长从而使得报文长度变长，一定程度上影响系统的处理效率。
2. 系统间随意命名空间定义，会使 ESB 在进行报文的转换过程中遇到问题，增加报文转换过程中的复杂度。
3. 过度集中的信息共享，会导致公共文件的改变导致大量的服务接口收到影响，增加编号带来的测试复杂度。

基于上述的要求和问题，所以需要进一步统一和规范 XML 命名空间在 WSDL 文件中的使用，避免出现较大的分歧。进行如下的约定（建议）：

1、 XML 的命名空间统一在 XML 根节点中进行定义。

2、 SOAP 规范所在的命名空间统一使用别名“soap”，对于 XML schema 的命名空间统一使用“xs”。

3、 浦发银行统一的数据字典（由数据标准组制定）存在一个 XSD 文件，该文件只定义数据字典的信息。命名空间统一使用“http://www.spdb.com.cn/spdb/metadata”，别名统一定义为“d”或是作为默认命名空间定义。

方式一：xmlns="http://www.spdb.com.cn/spdb/metadata"

方式二：xmlns:d="http://www.spdb.com.cn/spdb/metadata"

通过此种方式不仅可以达到信息共享的目的，也可以确保该文件一般情况只会增加信息，避免对修改过程中对大量服务产生不可以预期的影响。并要求数据字典的定义不能出现重复。

4、 具体服务接口结构统一定义在 wsdl 文件中或通过一个独立 XSD 文件定义，命名空间根据服务 ID 进行区分，但使用的别名为统一的别名。定义的样式形如：

xmlns:s="http://www.spdb.com.cn/spdb/service/[serviceid]"

serviceid 是 ESB 系统统一对外发布的服务 ID，全局唯一标识一个服务。

5、 服务消费方需要按照上述的规定进行服务的调用。

6、 服务提供方需要按照上述的规范进行服务的定义和发布。服务提供方在报文中定义的命名空间不能超过 2 个，并且别名的规则与 ESB 相同。

(四)通过 XML schema 定义和校验请求、输出的报文结构。

服务在 WSDL 中定义的数据结构明确定义了服务输入接口和输出接口，ESB 会根据该结构进行报文内容的校验和控制。主要包括几个方面的内容：

1、 ESB 系统会根据服务的接口生成校验的 XSD 文件对请求报文和响应报文进行校验。ESB 系统对报文的校验在开发测试环境下会完全校验，在生产环境会根据实际运行的情况选择关闭该校验。

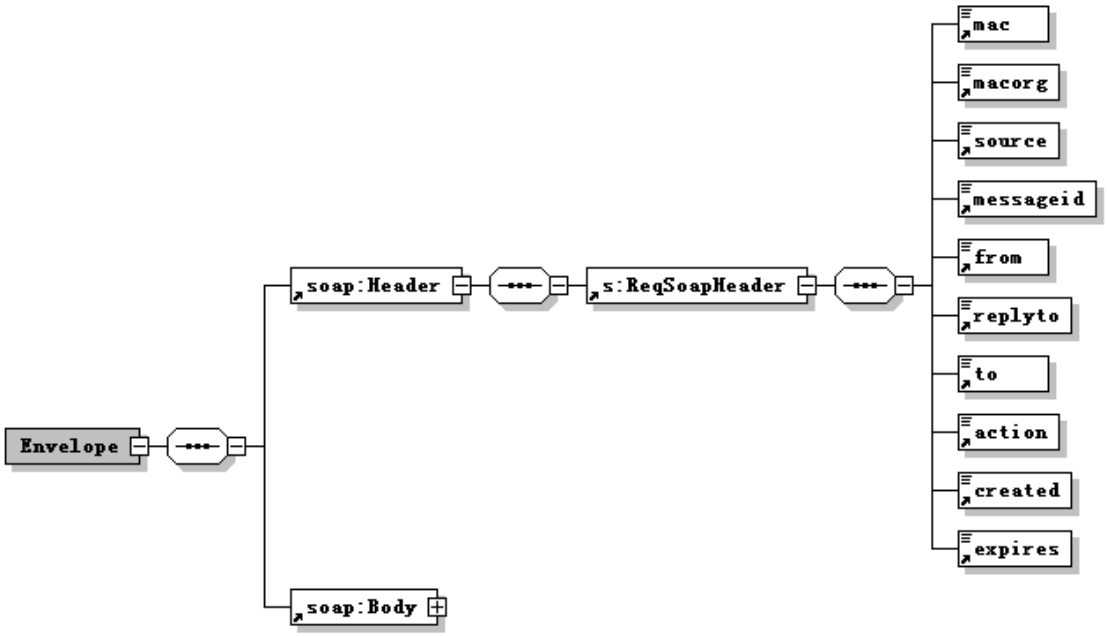
2、 服务接口的变更，如果请求字段增加一个非必需字段，则 ESB 可以在 XSD 文件中定义该字段为非必需的，其他请求方系统不需要修改。

3、 服务接口的变更，如果响应字段增加一个响应的字段，需要请求方确定该字段的响应不会影响到系统的调用。如果是采用框架生成的代码，该字段的返回会使得请求方校验报错而导致处理异常。所以对于新增响应的字段，要求服务提供方向下版本兼容，当服务上送的版本号大于等于该版本号时才输出该字段。或是服务提供方新增加一个版本信息，不同版本控制不同的输出。

4.1.1 SOAP 头的信息(Header)

4.1.1.1请求内容

SOAP 报文中 Header 的请求信息如下图所示：



序号	字段名称	字段名	类型	长度	必需	备注
1.	mac	MAC 字段	String	32	★	
2.	macorg	MAC 机构号	String	20	★	
3.	source	服务源端点地址	String	128	★	URI
4.	messageid	服务消息 ID	String	26	★	
5.	from	服务发起端点地址	String	128	★	URI
6.	replyto	服务响应端点地址	String	128	☆	URI
7.	to	服务地址	String	128	★	URI
8.	action	服务方法	String	128	★	URI
9.	created	消息创建时间	String	20	★	
10.	expires	消息超时时间	String	20	★	

字段说明：

- **mac**

报文的 MAC 字段，生成的方式是先用 32 个约定好的默认字符串初始化该字段，如“00000000000000000000000000000000”的初始字符串作为 MAC 字段初始值先组成待发送

的报文，通过该报文生成 16 个字节长度的 MD5 摘要，发送方调用加密平台的接口加密该 16 个字节的内容，生成 32 位 16 进制的 MAC 校验码，替换报文中的 MAC 初始字段后再发送给 ESB 系统。而 ESB 系统接收到请求方发过来的报文后，首先获取加密后的 MAC 字段，并且使用约定好的默认字符串替换报文中 MAC 校验码的内容后再生成 MD5 摘要。通过加密平台的接口加密 ESB 生成的摘要，比对接收到的 MAC 校验码和 ESB 系统生成的 MAC 校验码是不是完全相同，如果是一致则说明该报文是正常的报文，否则该报文为非法的报文。

ESB 需要控制是否对该服务消费者发送过来的消息进行 MAC 校验，如果需要校验则会进行 MAC 的校验；如果不需要校验服务消费者只要设置为初始字段的值即可。但此控制的开关是在 ESB 系统内部进行控制的。

遗留问题：在灾备切换的过程，可能存在跨机房没有进行 MAC 校验。

- **macorg**

加密时使用的虚拟机构号，要使用加密平台进行加解密处理，需要在加密平台申请一个机构号码，并且将该机构号所对应的密码设置到加密平台的硬件加密机内。调用加密平台时需要通过机构号来确定工作的密钥后通过该密钥进行相应的加密处理。

- **source**

服务源发起方系统，根据信息标准组制定的系统 ID 信息填写。例如一笔交易由 AMT 系统发起交易到 BIP 系统，而 BIP 系统又通过 ESB 系统请求核心，则该字段内容是 AMT 系统的系统 ID。如果系统无法取得源系统相关的内容，可以填发起系统的系统 ID，与 from 节点内容相同。

- **messageid**

消息的 ID 标识，编码规则是“6 位系统 ID+20 位序号”，要求系统实现该 ID 全局唯一。20 位序号是最长的要求，如果系统能够确保其唯一，比 20 字节短的字符也是可以合法处理。但要求其长度是固定的，不可部分消息的 messageid 是 10 位，部分消息却是 20 位。

- **from**

服务消费者系统 ID，根据信息标准组制定的系统 ID 信息填写。例如 ATM 系统请求 BIP 系统，而 BIP 系统又通过 ESB 系统请求核心系统，则该内容是 BIP 的系统 ID。

- **replyto**

服务响应端点地址。对于同步的系统调用，如 HTTP 协议同步请求的协议，该字段可以不填。对于异步或是有其他流程控制的消息处理，可以根据该字段将响应的信息路由给不同的系统。

可选字段，属于功能保留字段。

- **to**

服务地址，用于服务的寻址，填写的内容就是 ESB 服务的地址。

格式形如：http://www.spdb.com/esb/service/1001000100101

1001000100101 是服务编号，01 是服务的版本号。

- **action**

服务操作，通过该 URI 确定调用服务的操作方法。ESB 系统将通过 to 和 action 进行服务的寻址。

系统形如：http://www.spdb.com/esb/service/1001000100101/request/v100

- **created**

请求消息的创建时间，样式为“YYYYMMDD hh:mm:ss”，例如：2013-06-06 08:08:08。取自服务消费者系统的系统时间。

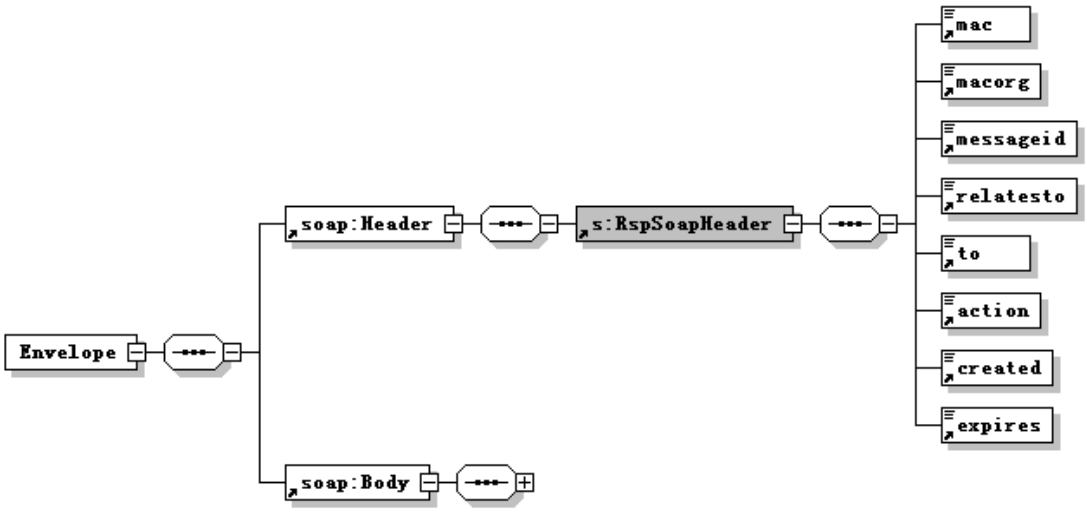
- **expires**

请求消息的超时时间，样式为“YYYYMMDD hh:mm:ss”，例如：2013-06-06 08:08:38。该时间需要比消息的创建时间大，ESB 系统将通过判断系统的时间与该时间的大小，如果 ESB 系统时间比该时间大，则 ESB 会拒绝此笔交易请求。可以有效防止消息重放攻击，使得消息在系统间正常流转。

注意：超时控制的前提是系统间有时钟同步，所以如果处于不同的机房，则时钟同步服务器会有所不同，有可能导致时间上差距较大。使用过程需要注意。

4.1.1.2响应内容

SOAP 报文头中响应的结构如下图所示：



序号	字段名称	字段名	类型	长度	必需	备注
1.	mac	MAC 字段	String	32	★	含义同请求
2.	macorg	MAC 机构号	String	6	★	含义同请求
3.	messageid	服务消息 ID	String	128	★	同步调用同请求，异步调用重新产生。
4.	relatesto	关联消息 ID	String	128	☆	同步调用可以不填，异步调用填请求消息 wsa:MessageID。
5.	to	服务目标系统号	String	128	★	填请求消息中的 wsa:ReplyTo/Address 相同。
6.	action	服务操作方法	String	128	★	
7.	created	消息创建时间	String	20	★	含义同请求
8.	expires	消息超时时间	String	20	★	含义同请求

可见响应的 Header 与请求的 Header 基本类似，少了部分字段，但增加 relatesto。需要注意的是 mac，macorg，created 和 expires 是根据响应消息重新产生，并不是请求报文中的内容。数据的处理方式与请求内容中的处理方式相同。

4.1.2 BODY 应用头信息（SHEAD）

4.1.2.1请求内容

为了兼容行内 SOP 现有的标准，并且支持新一代信息系统建设的信息交互，建议服务的公共头信息可以包含以下内容。

序号	字段名称	字段名	类型	最大长度	必需	备注
1.		PIN 种子	char(16)	16	☆	SOP 系统信息头（是否保留待定）
2.		密钥版本号	integer	4	☆	SOP 系统信息头（是否保留待定）
3.		终端号	String	5	☆	SOP 交易公共头
4.		城市代码	String	4	☆	SOP 交易公共头
5.		机构代码	String	4	★	SOP 交易公共头
6.		交易柜员	String	8	★	SOP 交易公共头
7.		交易流水号（前台流水号）	String	26	★	SOP 数据公共头
8.		交易日期	String	8	★	SOP 数据公共头
9.		交易时间	String	6	★	SOP 数据公共头
10.		授权柜员	String	8	☆	SOP 数据公共头
11.		授权密码	String	16	☆	SOP 数据公共头
12.		授权柜员有无卡标志	String	1	☆	SOP 数据公共头
13.		授权柜员卡序号	String	2	☆	SOP 数据公共头
14.		渠道系统 ID	String	10	☆	新增
15.		发起系统 ID	String	10	☆	新增
16.		全局流水号	String	26	☆	新增
17.		冲正日期	String	8	☆	新增
18.		冲正流水号	String	26	☆	新增

上述字段除了标志为必需一定要生成外，其他字段可以根据服务的具体需要来确定是否需要生成。不设置则该节点下不会在报文中出现。另外字段的先后顺序是没有关联的。

XML 结构如下图所示：

（待补）

原来 SOP 报文中包含系统信息头，公共交易头其中被屏蔽的交易头信息如下表所：

序号	字段名称	字段名	类型	最大长度	必需	备注
~~1.~~	~~SHJBCD~~	~~数据包长度~~	~~unsigned short~~	~~2~~		~~技术字段取消~~
~~2.~~	~~BAWMAC~~	~~报文 MAC~~	~~char(16)~~	~~16~~		~~SOAP 头定义~~
~~3.~~	~~MACJGH~~	~~MAC 机构号~~	~~char(4)~~	~~4~~		~~SOAP 头定义~~
~~4.~~	~~YNDIZH~~	~~目标地址~~	~~unsigned char(4)~~	~~4~~		~~技术字段取消~~
~~5.~~	~~MBIODZ~~	~~源地址~~	~~unsigned char(4)~~	~~4~~		~~技术字段取消~~
~~6.~~	~~BOLIUW~~	~~系统保留位~~	~~unsigned char(1)~~	~~1~~		~~暂不使用~~
~~7.~~	~~XXJSBZ~~	~~信息结束标志~~	~~unsigned char(1)~~	~~1~~		~~默认值为 1~~
~~8.~~	~~SJBSXH~~	~~报文序号~~	~~unsigned short~~	~~2~~		~~暂不使用~~
~~9.~~	~~JIOYBZ~~	~~校验标志~~	~~unsigned char(1)~~	~~1~~		~~暂不使用~~
~~10.~~		~~交易代码~~		~~4~~		
~~11.~~		~~交易子码~~		~~4~~		
~~12.~~		~~交易模式~~		~~1~~		
~~13.~~		~~交易序号~~		~~4~~		
~~14.~~		~~本交易包长度~~		~~4~~		
~~15.~~		~~系统偏移 1~~		~~2~~		
~~16.~~		~~系统偏移 2~~		~~2~~		

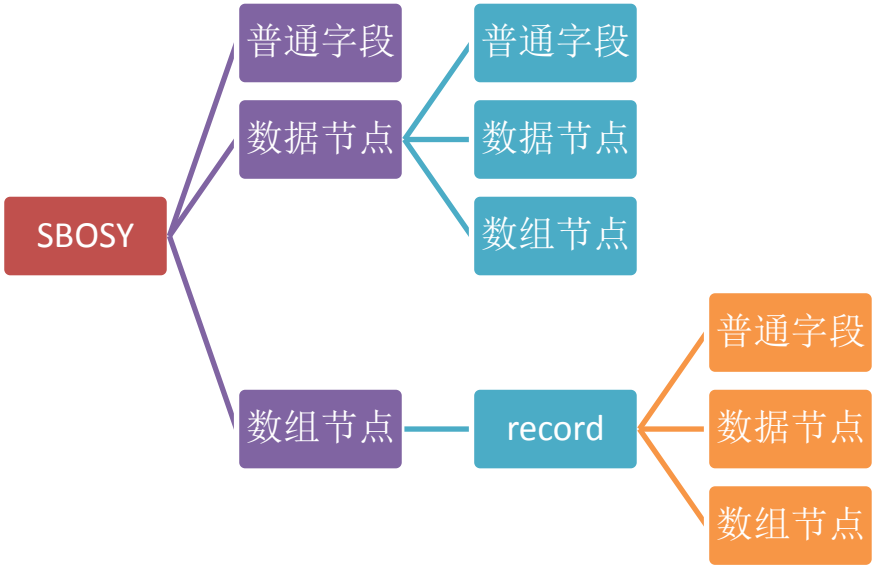
4.1.2.2响应内容

序号	字段名称	字段名	类型	最大长度	必需	备注
1.		交易代码	String	4	☆	
2.		联动交易码	String	4	☆	
3.		交易日期	String	8	★	
4.		交易时间	String	?	☆	
5.		柜员流水号	String	12	☆	
6.		出错交易序号	String	4	☆	
7.		错误代号	String	7	★	成功返回 AAAAAAA，其他返

						回错误代码
8.		错误信息	String	6	☆	
9.		错误提示	String	256	☆	
10.		全局流水号		26	☆	
11.		后台流水号		26	☆	
12.		后台系统 ID		10	☆	
13.		冲正原流水交易日期		8	☆	
14.		冲正原流水号		26	☆	

4.1.3 BODY 服务报文体（SBODY）

SBODY 下面可以有几种类型的数据：



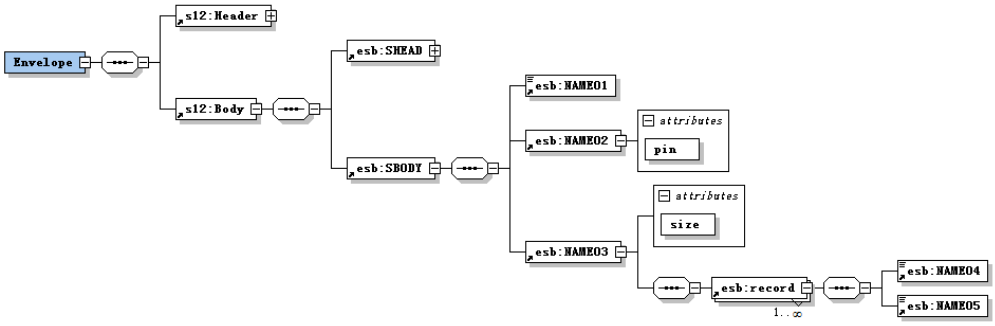
SBODY 底下可以存在普通字段，数据节点和数组字段。

普通字段：字段名及文本数据。形如：<A>value</A>

数据节点：字段名，作为 XML 的层级结构存在。形如：<A><B></B><C></C></A>

数组节点：数组节点的子节点是 recored，record 下面可以有各类数据节点，如普通字段、数据节点和数组节点，所以数组可以嵌套。

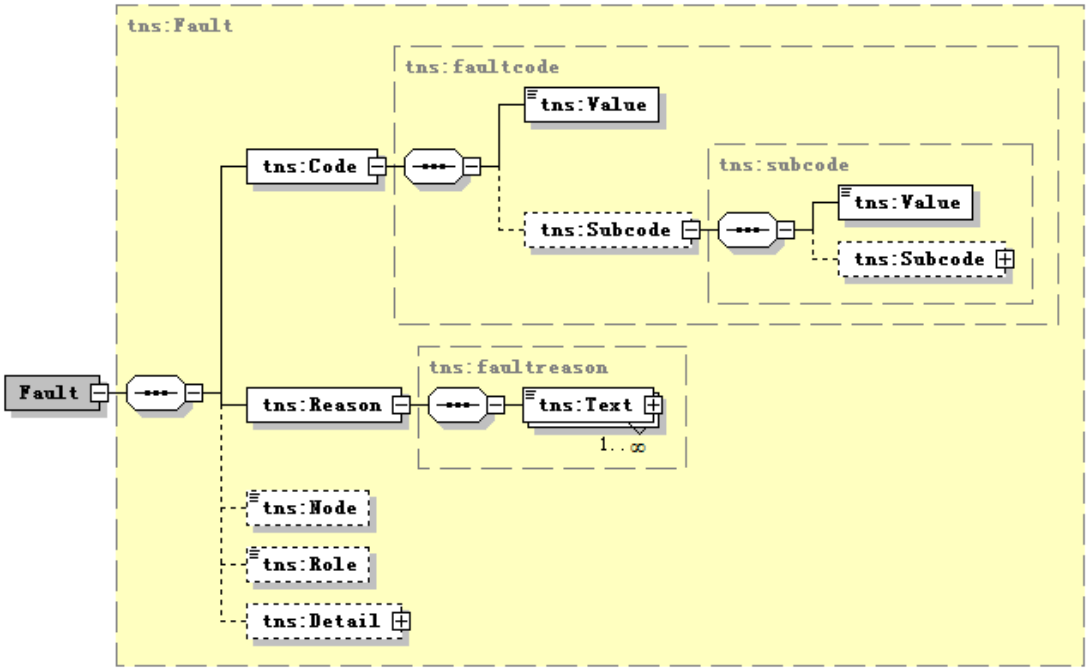
节点样例如下图所示：



SBODY 根据不同服务存在不同的内容，具体内容在服务接口进行详细描述，SOP 中的交易公共头会在此处进行定义。

4.1.4 系统异常返回（Fault）

ESB 系统在正常的情况下返回 SHEAD 和 SBODY 的节点，在 ESB 系统出现异常的情况下返回 Fault 节点。

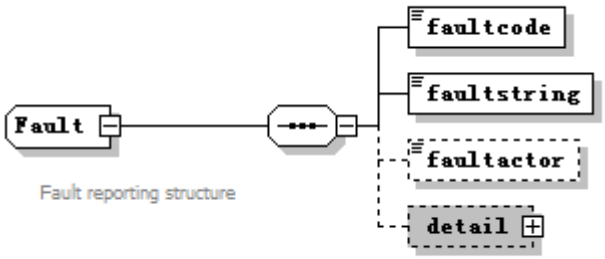


Code 填写的内容是：

错误码	错误信息	备注
DataEncodingUnknown	数据编码不可识别	
MustUnderstand	数据不可识别	
Receiver	接收出错	

Sender	发送出错	
VersionMismatch	版本不匹配	

Subcode 填写 ESB 系统内部的错误码，而 Reason 填写错误的原因和说明。



faultcode 填写的是 ESB 系统的错误码，faultstring 填写 ESB 系统的错误提示信息，其他字段默认为空。

4.1.5 通讯方式

SOAP 报文的标准通讯方式采用 HTTP1.1 的通讯协议，采用 POST 的方式进行消息的发送。不能采用 Keep-Alive。

4.1.6 报文编码

报文编码统一采用 UTF-8 的编码进行处理。

4.1.7 报文样例

4.1.7.1请求报文样例

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:soap-enc="http://schemas.xmlsoap.org/soap/encoding/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:x="http://www.w3.org/2001/XMLSchema"
xmlns:d="http://www.spdb.com.cn/spdb/metadata"
    xmlns:s="http://www.spdb.com.cn/spdb/service/service1">
    <soap:Header>
        <s:ReqHead>
```

```
            <d:mac>String</d:mac>
            <d:macorg>String</d:macorg>
            <d:source>String</d:source>
            <d:messageid>String</d:messageid>
            <d:from>String</d:from>
            <d:replyto>String</d:replyto>
            <d:to>String</d:to>
            <d:action>String</d:action>
            <d:expires>String</d:expires>
            <d:created>String</d:created>
        </s:ReqHead>
    </soap:Header>
    <soap:Body>
        <s:ReqScene01>
            <s:SHAED>
                <d:PINZHZ>String</d:PINZHZ>
                <d:MIYBBH>String</d:MIYBBH>
                <d:ZHNGDH>String</d:ZHNGDH>
                <d:CHSHDM>String</d:CHSHDM>
                <d:YNGYJG>String</d:YNGYJG>
                <d:JIO1GY>String</d:JIO1GY>
            </s:SHAED>
            <s:SBODY>
                <d:request_body1>String</d:request_body1>
                <d:request_body2>String</d:request_body2>
            </s:SBODY>
        </s:ReqScene01>
    </soap:Body>
</soap:Envelope>
```

4.1.7.2服务正常返回样例

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:soap-enc="http://schemas.xmlsoap.org/soap/encoding/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:x="http://www.w3.org/2001/XMLSchema"
xmlns:d="http://www.spdb.com.cn/spdb/metadata"
    xmlns:s="http://www.spdb.com.cn/spdb/service/service1">
    <soap:Header>
        <s:RspHead>
            <d:mac>String</d:mac>
            <d:macorg>String</d:macorg>
```

```
            <d:messageid>String</d:messageid>
            <d:relatesto>String</d:relatesto>
            <d:to>String</d:to>
            <d:action>String</d:action>
            <d:expires>String</d:expires>
            <d:created>String</d:created>
        </s:RspHead>
    </soap:Header>
    <soap:Body>
        <s:RspScene01>
            <s:SHAED>
                <d:JIAOYM>String</d:JIAOYM>
                <d:LDJYDM>String</d:LDJYDM>
                <d:JIOYRQ>String</d:JIOYRQ>
                <d:JIOYSJ>String</d:JIOYSJ>
                <d:GUIYLS>String</d:GUIYLS>
                <d:CWJYXH>String</d:CWJYXH>
                <d:PTCWDH>String</d:PTCWDH>
            </s:SHAED>
            <s:SBODY>
                <d:response_body1>String</d:response_body1>
                <d:response_body2>String</d:response_body2>
            </s:SBODY>
        </s:RspScene01>
    </soap:Body>
</soap:Envelope>
```

4.1.7.3服务错误返回样例

4.1.7.4ESB 异常返回样例

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
        <soap:Fault>
            <faultcode>E00101</faultcode>
            <faultstring>IP地址非法</faultstring>
            <detail/>
```

```
        </soap:Fault>
    </soap:Body>
</soap:Envelope>
```

4.1.7.5WSDL 定义样例

service1.wsdl 的样例（后面会有更新）：

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
    xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:http="http://schemas.xmlsoap.org/wsdl/http/"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/"
    xmlns:mime="http://schemas.xmlsoap.org/wsdl/mime/"
xmlns="http://www.spdb.com.cn/service1"
targetNamespace="http://www.spdb.com.cn/service1">
    <wsdl:types>
        <xsd:schema
xmlns:s="http://www.spdb.com.cn/spdb/service/service1"
attributeFormDefault="qualified"
            elementFormDefault="qualified"
targetNamespace="http://www.spdb.com.cn/service1">
            <xsd:import
namespace="http://www.spdb.com.cn/spdb/service/service1"
                schemaLocation="service1.xsd" />
            <xsd:element name="ReqHead" type="s:ReqHeadType"/>

            <xsd:element name="ReqScene01" type="s:ReqScene01Type"/>

            <xsd:element name="ReqScene02" type="s:ReqScene02Type"/>

            <xsd:element name="RspHead" type="s:RspHeadType"/>

            <xsd:element name="RspScene01" type="s:RspScene01Type"/>

            <xsd:element name="RspScene02" type="s:RspScene02Type"/>
        </xsd:schema>
    </wsdl:types>
    <wsdl:message name="ReqHead">
        <wsdl:part name="ReqHead" element="ReqHead" />
    </wsdl:message>
    <wsdl:message name="ReqScene01">
```

```
        <wsdl:part name="ReqScene01" element="ReqScene01" />
    </wsdl:message>
    <wsdl:message name="ReqScene02">
        <wsdl:part name="ReqScene02" element="ReqScene02" />
    </wsdl:message>
    <wsdl:message name="RspHead">
        <wsdl:part name="RspHead" element="RspHead" />
    </wsdl:message>
    <wsdl:message name="RspScene01">
        <wsdl:part name="RspScene01" element="RspScene01" />
    </wsdl:message>
    <wsdl:message name="RspScene02">
        <wsdl:part name="RspScene02" element="RspScene02" />
    </wsdl:message>
    <wsdl:portType name="ESBServerPortType">
        <wsdl:operation name="processScene1">
            <wsdl:input message="ReqScene01" />
            <wsdl:output message="RspScene01" />
        </wsdl:operation>
        <wsdl:operation name="processScene2">
            <wsdl:input message="ReqScene02" />
            <wsdl:output message="RspScene02" />
        </wsdl:operation>
    </wsdl:portType>
    <wsdl:binding name="ESBServerSoapBinding"
type="ESBServerPortType">
        <soap:binding style="document"
            transport="http://schemas.xmlsoap.org/soap/http" />
        <wsdl:operation name="processScene1">
            <soap:operation soapAction="urn:processScene1" />
            <wsdl:input>
                <soap:header message="ReqHead" part="ReqHead"
                    use="literal" />
                <soap:body use="literal" />
            </wsdl:input>
            <wsdl:output>
                <soap:header message="RspHead" part="RspHead"
                    use="literal" />
                <soap:body use="literal" />
            </wsdl:output>
        </wsdl:operation>

        <wsdl:operation name="processScene2">
            <soap:operation soapAction="urn:processScene2" />
```

```
            <wsdl:input>
                <soap:header message="ReqHead" part="ReqHead"
                    use="literal" />
                <soap:body use="literal" />
            </wsdl:input>
            <wsdl:output>
                <soap:header message="RspHead" part="RspHead"
                    use="literal" />
                <soap:body use="literal" />
            </wsdl:output>
        </wsdl:operation>
    </wsdl:binding>
    <wsdl:service name="ESBServer">
        <wsdl:port name="ESBServerSoapEndpoint"
binding="ESBServerSoapBinding">
            <soap:address
location="http://127.0.0.1:8080/demo/services/ESBServer" />
        </wsdl:port>
    </wsdl:service>
</wsdl:definitions>
```

涉及文件 service1.xsd 的样例：

```
<?xml version="1.0" encoding="UTF-8"?>
<x:schema xmlns:x="http://www.w3.org/2001/XMLSchema"
xmlns:s="http://www.spdb.com.cn/spdb/service/service1"
xmlns:d="http://www.spdb.com.cn/spdb/metadata"
targetNamespace="http://www.spdb.com.cn/spdb/service/service1"
elementFormDefault="qualified" attributeFormDefault="qualified">
    <x:import namespace="http://www.spdb.com.cn/spdb/metadata"
schemaLocation="metadata.xsd"/>
    <!-- request head -->
    <x:element name="ReqHead" type="s:ReqHeadType"/>
    <x:complexType name="ReqHeadType">
        <x:sequence>
            <x:element ref="d:mac" minOccurs="0"/>
            <x:element ref="d:macorg" minOccurs="0"/>
            <x:element ref="d:source" minOccurs="0"/>
            <x:element ref="d:messageid" minOccurs="0"/>
            <x:element ref="d:from" minOccurs="0"/>
            <x:element ref="d:replyto" minOccurs="0"/>
            <x:element ref="d:to" minOccurs="0"/>
            <x:element ref="d:action" minOccurs="0"/>
            <x:element ref="d:expires" minOccurs="0"/>
            <x:element ref="d:created" minOccurs="0"/>
```

```
        </x:sequence>
    </x:complexType>
    <!-- request body scene1 -->
    <x:element name="ReqScene01" type="s:ReqScene01Type"/>
    <x:complexType name="ReqScene01Type">
        <x:sequence>
            <x:element name="SHAED">
                <x:complexType>
                    <x:sequence>
                        <x:element ref="d:PINZHZ" minOccurs="0"/>
                        <x:element ref="d:MIYBBH" minOccurs="0"/>
                        <x:element ref="d:ZHNGDH" minOccurs="0"/>
                        <x:element ref="d:CHSHDM" minOccurs="0"/>
                        <x:element ref="d:YNGYJG" minOccurs="0"/>
                        <x:element ref="d:JIO1GY" minOccurs="0"/>
                    </x:sequence>
                </x:complexType>
            </x:element>
            <x:element name="SBODY">
                <x:complexType>
                    <x:sequence>
                        <x:element ref="d:request_body1" minOccurs="0"/>
                        <x:element ref="d:request_body2" minOccurs="0"/>
                    </x:sequence>
                </x:complexType>
            </x:element>
        </x:sequence>
    </x:complexType>
    <!-- request body scene2 -->
    <x:element name="ReqScene02" type="s:ReqScene02Type"/>
    <x:complexType name="ReqScene02Type">
        <x:sequence>
            <x:element name="SHAED">
                <x:complexType>
                    <x:sequence>
                        <x:element ref="d:PINZHZ" minOccurs="0"/>
                        <x:element ref="d:MIYBBH" minOccurs="0"/>
                        <x:element ref="d:ZHNGDH" minOccurs="0"/>
                        <x:element ref="d:CHSHDM" minOccurs="0"/>
                        <x:element ref="d:YNGYJG" minOccurs="0"/>
                        <x:element ref="d:JIO1GY" minOccurs="0"/>
                    </x:sequence>
                </x:complexType>
            </x:element>
```

```
            <x:element name="SBODY">
                <x:complexType>
                    <x:sequence>
                        <x:element ref="d:request_body1" minOccurs="0"/>
                        <x:element ref="d:request_body2" minOccurs="0"/>
                    </x:sequence>
                </x:complexType>
            </x:element>
        </x:sequence>
    </x:complexType>
    <!--

    ################################################################
#################################################
    -->
    <!-- response head -->
    <x:element name="RspHead" type="s:RspHeadType"/>
    <x:complexType name="RspHeadType">
        <x:sequence>
            <x:element ref="d:mac" minOccurs="0"/>
            <x:element ref="d:macorg" minOccurs="0"/>
            <x:element ref="d:messageid" minOccurs="0"/>
            <x:element ref="d:relatesto" minOccurs="0"/>
            <x:element ref="d:to" minOccurs="0"/>
            <x:element ref="d:action" minOccurs="0"/>
            <x:element ref="d:expires" minOccurs="0"/>
            <x:element ref="d:created" minOccurs="0"/>
        </x:sequence>
    </x:complexType>
    <!-- response body scene1 -->
    <x:element name="RspScene01" type="s:RspScene01Type"/>
    <x:complexType name="RspScene01Type">
        <x:sequence>
            <x:element name="SHAED">
                <x:complexType>
                    <x:sequence>
                        <x:element ref="d:JIAOYM" minOccurs="0"/>
                        <x:element ref="d:LDJYDM" minOccurs="0"/>
                        <x:element ref="d:JIOYRQ" minOccurs="0"/>
                        <x:element ref="d:JIOYSJ" minOccurs="0"/>
                        <x:element ref="d:GUIYLS" minOccurs="0"/>
                        <x:element ref="d:CWJYXH" minOccurs="0"/>
                        <x:element ref="d:PTCWDH" minOccurs="0"/>
                    </x:sequence>
```

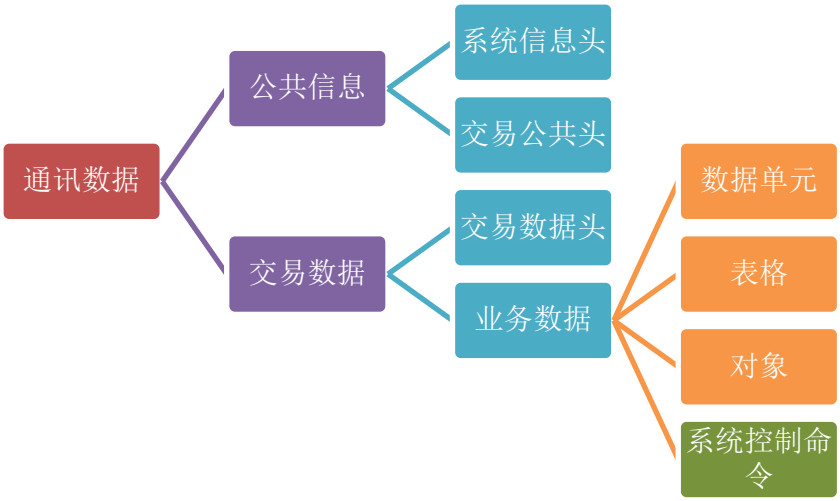
```
                </x:complexType>
            </x:element>
            <x:element name="SBODY">
                <x:complexType>
                    <x:sequence>
                        <x:element ref="d:response_body1" minOccurs="0"/>
                        <x:element ref="d:response_body2" minOccurs="0"/>
                        <x:any namespace="d" processContents="lax"
minOccurs="0" maxOccurs="unbounded"/>
                    </x:sequence>
                </x:complexType>
            </x:element>
        </x:sequence>
    </x:complexType>
    <!-- response body scene2 -->
    <x:element name="RspScene02" type="s:RspScene02Type"/>
    <x:complexType name="RspScene02Type">
        <x:sequence>
            <x:element name="SHAED">
                <x:complexType>
                    <x:sequence>
                        <x:element ref="d:JIAOYM" minOccurs="0"/>
                        <x:element ref="d:LDJYDM" minOccurs="0"/>
                        <x:element ref="d:JIOYRQ" minOccurs="0"/>
                        <x:element ref="d:JIOYSJ" minOccurs="0"/>
                        <x:element ref="d:GUIYLS" minOccurs="0"/>
                        <x:element ref="d:CWJYXH" minOccurs="0"/>
                        <x:element ref="d:PTCWDH" minOccurs="0"/>
                    </x:sequence>
                </x:complexType>
            </x:element>
            <x:element name="SBODY">
                <x:complexType>
                    <x:sequence>
                        <x:element ref="d:response_body3" minOccurs="0"/>
                        <x:element ref="d:response_body4" minOccurs="0"/>
                        <x:any namespace="d" processContents="lax"
minOccurs="0" maxOccurs="unbounded"/>
                    </x:sequence>
                </x:complexType>
            </x:element>
        </x:sequence>
    </x:complexType>
</x:schema>
```

涉及数据字典定义文件 metadata.xsd 的文件样例如：

```
<?xml version="1.0" encoding="UTF-8"?>
<x:schema xmlns:x="http://www.w3.org/2001/XMLSchema"
xmlns:tns="http://www.spdb.com.cn/spdb/metadata"
targetNamespace="http://www.spdb.com.cn/spdb/metadata"
elementFormDefault="qualified" attributeFormDefault="qualified">
    <!-- request and response soapheader -->
    <x:element name="mac" type="x:string"/>
    <x:element name="macorg" type="x:string"/>
    <x:element name="source" type="x:string"/>
    <x:element name="messageid" type="x:string"/>
    <x:element name="from" type="x:string"/>
    <x:element name="replyto" type="x:string"/>
    <x:element name="to" type="x:string"/>
    <x:element name="action" type="x:string"/>
    <x:element name="expires" type="x:string"/>
    <x:element name="created" type="x:string"/>
    <x:element name="relatesto" type="x:string"/>
    <!-- request shead -->
    <x:element name="PINZHZ" type="x:string"/>
    <x:element name="MIYBBH" type="x:string"/>
    <x:element name="ZHNGDH" type="x:string"/>
    <x:element name="CHSHDM" type="x:string"/>
    <x:element name="YNGYJG" type="x:string"/>
    <x:element name="JIO1GY" type="x:string"/>
    <!-- request sbody -->
    <x:element name="request_body1" type="x:string"/>
    <x:element name="request_body2" type="x:string"/>
    <x:element name="request_body3" type="x:string"/>
    <x:element name="request_body4" type="x:string"/>
    <!-- response shead -->
    <x:element name="JIAOYM" type="x:string"/>
    <x:element name="LDJYDM" type="x:string"/>
    <x:element name="JIOYRQ" type="x:string"/>
    <x:element name="JIOYSJ" type="x:string"/>
    <x:element name="GUIYLS" type="x:string"/>
    <x:element name="CWJYXH" type="x:string"/>
    <x:element name="PTCWDH" type="x:string"/>
    <!-- response sbody -->
    <x:element name="response_body1" type="x:string"/>
    <x:element name="response_body2" type="x:string"/>
    <x:element name="response_body3" type="x:string"/>
    <x:element name="response_body4" type="x:string"/>
</x:schema>
```

4.2 SOP 报文规范

4.2.1 概述



通讯数据包由**公共信息**部分、**交易数据**部分

公共信息部分包括**系统信息头**和**交易公共信息头**。

交易数据部分包括**交易数据头（可选）**、**业务数据**和**系统控制命令**。

业务数据部分又包括**数据单元**、**表格**和**对象**。业务数据部分可以插入系统控制命令。

- **系统信息头**

部分是与系统相关的，包括目标服务代码、数据源代码、长度等系统信息，由系统配置文件（数据库表，下同）指定，每字段的个数、顺序、长度固定。

- **交易公共信息头**

部分是跟同一数据包中所包含的所有交易相关的信息，包括交易的终端，交易柜员，机构等信息，组织方式同系统信息头的格式。

- **交易数据头**

部分包括交易代码，交易模式，前台流水号,授权柜员等信息，组织方式同系统信息头的格式。

- **业务数据部分**

由长度+数据组成，长度用一个字节的二进制数表示，数据全部转换成字符串传送，字符串的最大长度由系统的 BYTE_MAX_LEN 宏定义指定(在本系统里，我们定义 BYTE_MAX_LEN

为 250（0xFA）），大于该长度的部分系统保留作为控制命令用。如果数据单元的长度超过 BYTE_MAX_LEN，则分段发送，0xFF 作为超长数据的标志。例如要传 768 字节数据，由于系统定义 BYTE_MAX_LEN 为 250，则该数据段为 0xFF+250 字符+0xFF+250 字符+0xFF+250 字符+0x12+18 字符；如要传 250 字节数据时，则该数据段为 0xFA+250 字符。

控制命令部分。在业务数据中，可以穿插控制命令，其格式为：控制字符标志+控制字符串长度+控制字符串。控制字符标志由系统保留的 BYTE_MAX_LEN~0xFF 之间的字符标识，在本系统中系统保留字符为 0xFB，0xFC，0xFD，0xFE，目前仅使用 0XFE 作为打印相关的控制命令标识符。

整个通讯数据包的组成示意图如下：

<table>
<tr><td colspan="2">公共信息</td><td colspan="7">交易数据</td></tr>
<tr><td colspan="2"></td><td>交易头</td><td colspan="6">业务数据</td></tr>
<tr><td>系统信息头</td><td>交易公共信息头</td><td>交易数据头</td><td>数据单元</td><td>数据单元</td><td>数据单元</td><td>表格单元</td><td>对象单元</td><td>。。。</td></tr>
<tr><td colspan="9">控制命令码可以穿插在业务数据之间，如单元与单元之间，表格内部数据项之间，以及对象内数据项之间</td></tr>
</table>

下面将分几个部分详细描述整个系统的通讯格式。

4.2.2 单元定义

4.2.2.1数据单元（TRANFLD）

数据单元 TRANFLD 是 COP 和 SOP 中代表特定内容的基本数据项，一般对应一个应用数据定义，在 SOP 平台中采用可自解包的通讯格式。数据单元通讯格式如下：

<table>
<tr><td>可选</td><td colspan="2">机构名</td><td>可选</td><td colspan="2">金额</td><td>可选</td><td colspan="4">266 字节数据</td><td colspan="2">。。。</td></tr>
<tr><td>属性</td><td>0X5</td><td>“A 网点”</td><td>属性</td><td>0X 6</td><td>“123.45”</td><td>属性</td><td>0xFF</td><td>250 字节</td><td>0X10</td><td>16 字节</td><td></td><td></td></tr>
</table>

每一个数据单元在通讯格式中用两个部分表示——长度+内容。内容部分都以字符串方式传输，截掉前导和后续的空格，以减少冗余数据的传输。长度以一个字节的 16 进制数表示，可表示的最大长度为 250（0XFA）字节，若数据单元长度超过 250 字节，则采用分解传送的方式，以 0XFF 表示数据单元超长，如上图中 266 字节数据单元。

长度定义中 0XFB~0xFE 保留，作为控制命令使用，见控制命令描述。

属性是可选项，可以包括多种属性，目前仅支持打印属性。对于一般数据单元不需要定

义，对于其它对象一般需要定义相应属性。

打印属性定义格式如下：

打印行	打印列	打印模式
Y	X	0，1，2...

本系统中，要求 Y<=250 && X<=250。

打印模式的具体含义见打印模式定义。

4.2.2.2表格单元（FORM）

表格单元 FORM 是指 COP 和 SOP 平台中由格式相同的多条记录组成的复合数据单元，其中每一条记录的数据又由多个数据单元组成。表格在通讯格式中以表格名+记录条数+多条记录数据组成，每一条记录又由多个数据单元（表示方法同数据单元）表示。

表格单元的通讯格式如下：

表格名		记录条数	栏位个数	可选项	记录 01						记录 02					
0X3	“F01”	0X2	0X3	打印属性或其它	0X3	“ABC”	0X2	借	0x 7	“1234.99”	0X4	“李明”	0X2	贷	0x5	“20.00”
表格名长度	表格名称			见下表	姓名项		借贷标志项		金额项		姓名项		借贷标志项		金额项	

属性是可选项，在一般数据单元的定义中不使用。

在本系统中，记录条数不得大于 250 个。如果记录条数可能大于 250 条，必须采用文件传输的方式解决这一问题。

表格的打印属性定义如下：

起始行号	打印列 1	模式 1	打印列 2	模式 2	...	打印列 n	模式 n
Y	X1	M1	X2	M2		Xn	Mn

模式 M1~Mn 为数字代码，含义同数据单元中的说明。

其它属性定义可以根据扩展对象的属性进行定义。

4.2.2.3对象单元

COP/SOP 的对象代表一个具有特定属性的复合数据单元，是 COP/SOP 系统实现后台驱动前台的基础。对象描述中必须包括对象名，其它属性如字段排列顺序等可在外部静态定义。

对象的组成元素可以包括数据单元和表格单元等系统支持的各类单元。

在 COP/SOP 系统中，对象的类型可以根据需求进行扩展和定义，目前支持的对象类型包括窗口对象、打印对象等。

对象一般由对象名+数据单元(可选项)+表格单元(可选项)组成。一个包含数据单元和表格单元的窗口对象的通讯格式如下：

对象名		对象内容					
0X5	“OBJ01”	数据单元	数据单元	。。。	表格单元 01	数据单元	。。。

窗口对象的对象内容中不包含数据单元和表格单元的可选属性，如行、列、模式等信息。

打印对象由对象名+［打印属性＋数据单元］(可选项)+［表格名称＋记录条数＋栏位数＋打印属性＋格数据］(可选项)＋打印控制命令码（可选项）组成。打印控制命令码见打印控制命令码介绍。

4.2.2.4控制命令码

控制命令码可以穿插在数据单元之间，也可以穿插在表格单元的数据项之间。COP/SOP 系统中保留了 0XFB~0XFE 之间的取值作为控制命令标志用。目前仅使用控制命令标志 0XFE，作为打印控制命令码传输标志。

控制命令标志	控制命令长度	N 位控制序列
0XFE	0xN	0x14。。。

在这里有一个换页打印的问题，本系统定义了 0xFE 为系统控制命令标志，换页符为 0x0c 换行 0x0a。因此，如果需要换页打印，该换页控制命令串为 0xFE+0x01+0x14；该串可以置于打印对象的数据单元或表格单元之前（如 0xFE+0x01+0x14+TRANFLD 或 FORM），也可以置于 FORM 数据中某一条记录之前（如 FORM 的第 n-1 条记录+0xFE+0x01+0x14+FORM 的第 n 条记录），其它地方一概予以忽略。

4.2.3 输入信息格式

输入 / 输出是相对后台主机而言的。输入信息是指客户端发向后台主机的信息。输入信息由系统信息头、交易公共头、交易数据头、业务数据组成。

其中系统信息头、交易公共头和交易数据头采用结构化的数据格式；业务数据采用长度

加字符串的传输方式，在本系统中，除二进制数据外，其他数据都转换成字符串类型进行传输。

输入信息包的格式如下：

公共信息		交易数据 1					交易数据 2				。。。
系统信息头	交易公共头	交易数据头	数据单元	数据单元	表格单元	。。。	交易数据头	数据单元	数据单元	。。。	

输入信息格式中不允许存在对象单元和控制命令码，业务数据由数据单元和表格单元组成。数据单元和表格单元的传输格式见单元定义。

下面将介绍系统信息头、交易公共头、交易数据头三个部分的数据格式。

4.2.3.1系统信息头

系统信息头的格式如下：（55bytes） 报文头总长 140B

{

数据包长度 （SHJBCD） unsigned short 2B //转换成网络格式

报文 MAC (BAWMAC) char(16)

MAC机构号 MACJGH char(4) //表示报文中MAC的机构号

PIN种子 PINZHZ char(16) //表示报文中PIN的种子(随机数参与加密)

目标地址 (YNDIZH) unsigned char(4) //二进制方式存储

源地址 (MBIODZ) unsigned char(4) //二进制方式存储

系统保留位 (BOLIUW) unsigned char(1) //二进制方式存储

信息结束标志(XXJSBZ) unsigned char(1) //二进制方式存储

报文序号 (SJBSXH) unsigned short 2B //转换成网络格式

校验标志 (JIOYBZ) unsigned char(1) //二进制方式存储

密钥版本号 (MIYBBH) integer 4B

}

字段说明

信息长度：表示整个数据包的长度，包括本数据项所占用的字节数。

目标地址：目标服务器的地址，以代码表示，包含目标服务器，服务端口等信息。具体表示方法见附录。

源地址：交易发起端地址，以代码表示，包含源设备，服务端口等信息。具体表示方法

见附录。

系统保留位：暂不使用。默认设置为 0

信息结束标志：为适应多包数据通讯保留，默认设置为 1

报文序号：为了防止报文恶意重发设置，暂不使用，默认值为 0

校验标志：是否需要 MAC 或其它校验，暂不使用;

报文 MAC：　校验算法生成的数据项

4.2.3.2交易公共头

公共交易头格式如下：(21bytes)

```
{
终端号      ZHNGDH  char(5)      String    //字符串方式存储
城市代码    CHSHDM  char(4)      String
机构代码    YNGYJG  char(4)      String
交易柜员    JIO1GY      char(8)      String
}
```

字段说明

终端号：交易发起端终端号，省略 TTY，如 21a 表示终端 tty21a。

城市代码：暂不使用，设置为空格

机构代码：银行机构代码。

交易柜员：交易柜员代码。

4.2.3.3交易数据头

交易数据头格式如下：(64bytes)

```
{
交易代码    JIAOYM      char(4)             //String
交易子码    JIOYZM      char(2)             //String
交易模式    JIOYMS      char(1)             //String
交易序号    JIOYXH      int       4B        //int   (pjyjk: N 4)
本交易包长度 COMMLN     unsigned short        2B   (pjyjk:n 2)//
系统偏移 1  PNYIL1      unsigned short        2B //0xFFFF 为无效
系统偏移 2  PNYIL2      unsigned short        2B //0xFFFF 为无效
前台流水号  QANTLS      char(12)
前台日期    QANTRQ  char(8)
授权柜员    SHOQGY  char(8)             //String
授权密码    SHOQMM      char(16)            //String
```

授权柜员有无卡标志　YWKABZ　char(1)
授权柜员卡序号　CZYNXH　char(2)
}

字段说明

交易子码：　判断某个复合交易中的子交易码

交易序号：主要为组合交易使用，是前台带到后台的组合交易序号。

系统偏移：记录加密字段的偏移位置

授权柜员有无卡标志：0-无卡 1-有卡；

4.2.4 输出信息格式

输出信息是指后台交易完成后返回给前台的通讯数据，由系统信息头、交易公共头、返回业务数据组成。

输出信息包的格式如下：

公共信息		业务数据			
系统信息头	交易公共头	对象 1	对象 2	。。。	对象 n

4.2.4.1系统信息头

系统信息头同输入信息格式。

4.2.4.2交易公共头

交易公共头的数据格式如下：(41bytes)

```
{
交易代码        JIAOYM        unsigned char(4)
联动交易码      LDJYDM       unsigned  char(4)        /*--前台读取—前台再联动-*/
交易日期        JIOYRQ        char(8)
交易时间        JIOYSJ        int   4B (PJYJK : L   4)
柜员流水号      GUIYLS        char(12)
出错交易序号    CWJYXH  short      2B
错误代号        PTCWDH  char(7)   //成功返回 AAAAAAA，其他返回错误代码
}
```

字段描述

交易代码：　输入数据中的第一个交易代码。

柜员流水号：财务交易产生，包含四位柜员号及六位顺序号信息。

出错交易序号：出错交易的交易序号，是由前台生成并传输到后台的。

交易状态：　前三位为错误类型，后 4 位为错误代码。见附录错误代码分类表。

4.2.4.3业务数据部分

对于业务数据部分，采用对象传输方式，不允许直接传输单独的数据单元或表格单元。

4.2.4.3.1　错误信息对象

在错误情况下，此处为错误描述对象，其格式如下例：

错误信息对象名		错误信息	
0X06	“ERRMSG”	0X02	-1　0x　“此帐号不存在”

4.2.4.3.2　正常交易对象

交易状态正常情况下，数据格式如下：

业务数据			
对象 1	对象 2	。。。	对象 N

4.2.5 打印模式定义

打印模式	字符代码	含义
0（默认）	S4c1	宋体，正常字，15CPI，6LPI
1	S4c1	宋体，正常字，15CPI，6LPI
2	S4C1	宋体，正常字，10CPI，6LPI
3	S4c2	宋体，正常字，15CPI，8LPI
4	S4C2	宋体，正常字，10CPI，8LPI
5	S4c3	宋体，正常字，15CPI，10LPI
6	S4C3	宋体，正常字，10CPI，10LPI

7	S4c4	宋体，正常字，15CPI，12LPI
8	S4C4	宋体，正常字，10CPI，12LPI
9	S4c5	宋体，正常字，15CPI，4LPI
10	S4C5	宋体，正常字，10CPI，4LPI
11	S3c1	宋体，倍高字，15CPI，6LPI
12	S3C1	宋体，倍高字，10CPI，6LPI
13	S3c2	宋体，倍高字，15CPI，8LPI
14	S3C2	宋体，倍高字，10CPI，8LPI
15	S3c3	宋体，倍高字，15CPI，10LPI
16	S3C3	宋体，倍高字，10CPI，10LPI
17	S3c4	宋体，倍高字，15CPI，12LPI
18	S3C4	宋体，倍高字，10CPI，12LPI
19	S3c5	宋体，倍高字，15CPI，4LPI
20	S3C5	宋体，倍高字，10CPI，4LPI
21	S2c1	宋体，倍宽字，15CPI，6LPI
22	S2C1	宋体，倍宽字，10CPI，6LPI
23	S2c2	宋体，倍宽字，15CPI，8LPI
24	S2C2	宋体，倍宽字，10CPI，8LPI
25	S2c3	宋体，倍宽字，15CPI，10LPI
26	S2C3	宋体，倍宽字，10CPI，10LPI
27	S2c4	宋体，倍宽字，15CPI，12LPI
28	S2C4	宋体，倍宽字，10CPI，12LPI
29	S2c5	宋体，倍宽字，15CPI，4LPI
30	S2C5	宋体，倍宽字，10CPI，4LPI
31	S1c1	宋体，倍宽倍高字，15CPI，6LPI
32	S1C1	宋体，倍宽倍高字，10CPI，6LPI
33	S1c2	宋体，倍宽倍高字，15CPI，8LPI
34	S1C2	宋体，倍宽倍高字，10CPI，8LPI
35	S1c3	宋体，倍宽倍高字，15CPI，10LPI
36	S1C3	宋体，倍宽倍高字，10CPI，10LPI
37	S1c4	宋体，倍宽倍高字，15CPI，12LPI
38	S1C4	宋体，倍宽倍高字，10CPI，12LPI
39	S1c5	宋体，倍宽倍高字，15CPI，4LPI
40	S1C5	宋体，倍宽倍高字，10CPI，4LPI

4.2.6 通讯方式

通讯方式采用 Socket 短连接的方式进行处理。

4.2.7 报文编码

SOP 报文的报文编码是 GBK。

4.2.8 报文样例

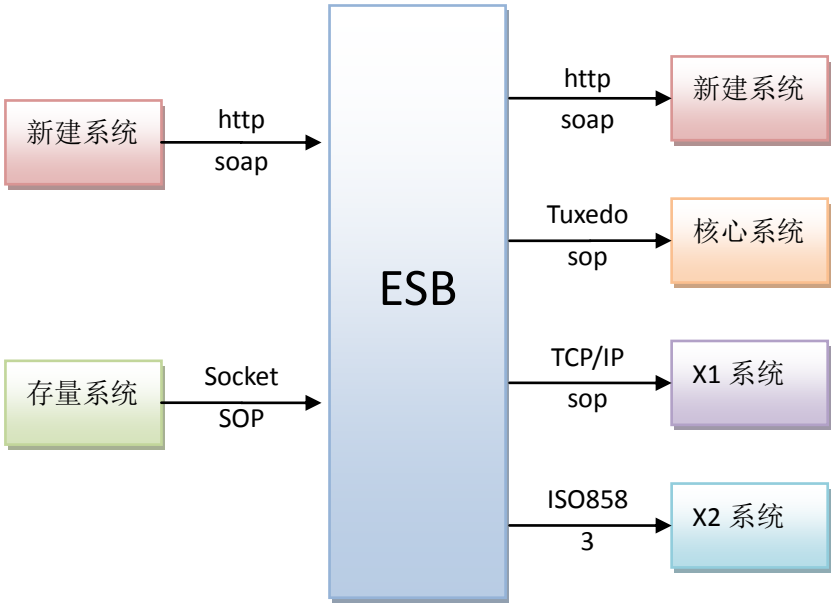
5 系统集成方法

5.1 系统集成准则

浦发银行 ESB 系统对服务消费者（即请求 ESB 服务的系统）和服务服务提供者（即通过 ESB 系统发布服务的系统）的集成原则如下：

系统类别	系统状态	集成准则
服务消费者（请求方）	存量系统	SOCKET+SOP+GBK 或 HTTP+SOAP+UTF-8。二选其一。
	新建系统	HTTP+SOAP+UTF-8。
服务提供者（提供方）	存量系统	ESB 适配集成存量系统，提供方不作过多修改。
	新建系统	HTTP+SOAP+UTF-8。

系统集成的简单逻辑示意图：



所以在前端只有两种通讯的方式 HTTP+SOAP 和 Socket+SOP。而后台服务提供者系统如果是新建系统要求一定按照 ESB 确定的标准进行服务的发布，存量系统将会保持原有的方式不变进行服务的发布，由 ESB 系统进行适配处理。

存量服务消费者可以分批分阶段逐步切换到 ESB 系统中去。

5.2 系统编码准则

ESB 系统将根据 SOAP 报文 Header 中 wsa:from/wsa:Address 和 esb:source/wsa:Address 的内容提取服务发起系统和服务源发起系统的系统 ID，该系统 ID 需要根据数据标准组制定的系统 ID 进行填写。ESB 将根据此内容进行权限控制、流量控制、IP 地址控制、流水登记、运行监控等。

5.3 系统安全控制准则

ESB 系统对服务的消费者和服务的提供者提供安全的控制，控制的内容包括如下几个方面：

5.3.1 IP 地址控制

ESB 系统将根据 SOAP 报文 Header 中 wsa:from/wsa:Address 的内容提取服务发起系统的系统 ID。ESB 系统将检查系统 ID 和发起方的 IP 地址是否有授权，存在授权则交易正常处理；不存在授权或授权为拒绝接收，则交易会被直接拒绝。所以系统要调用 ESB，需要提交请求 ESB 系统的客户端 IP 地址。

5.3.2 服务权限控制

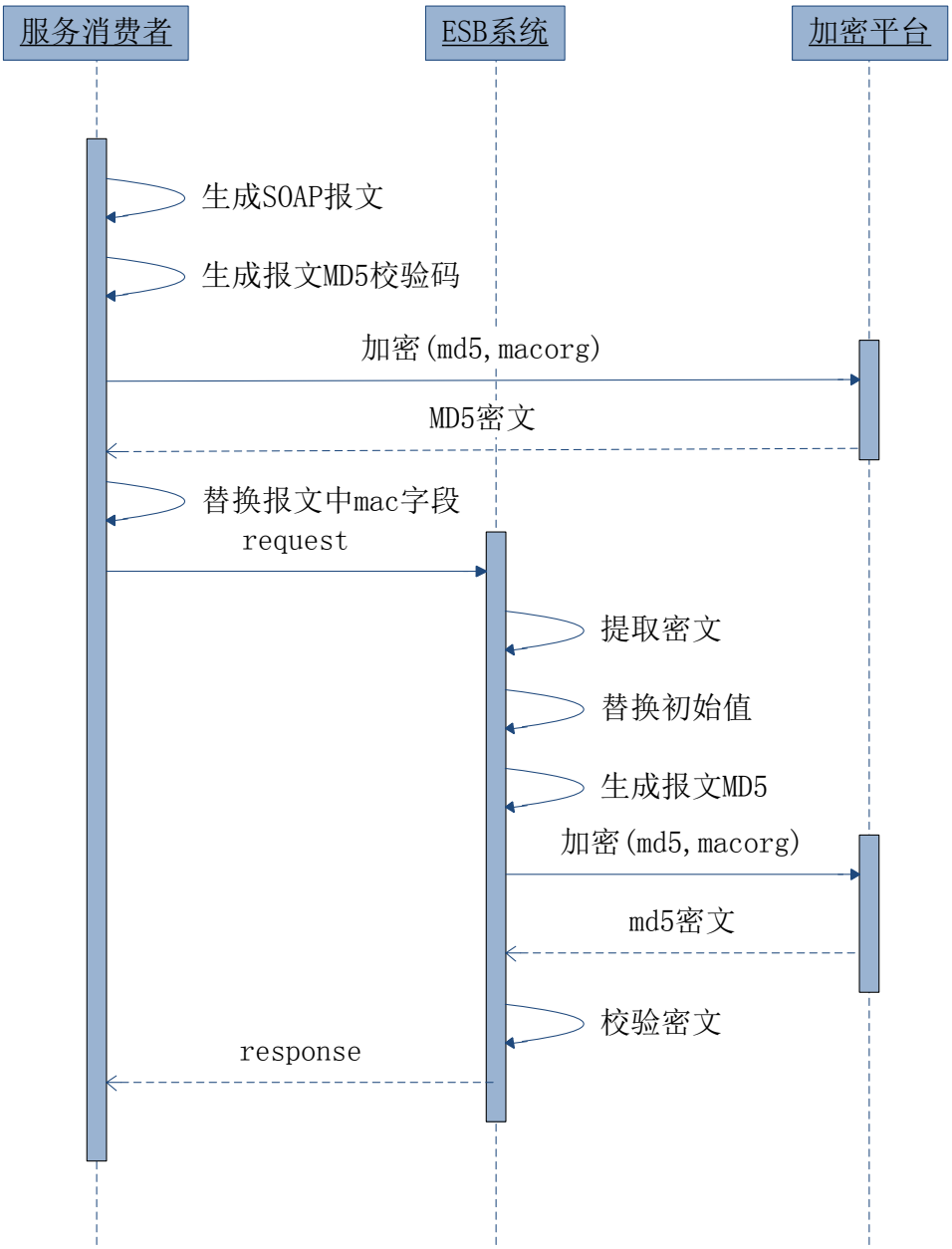
ESB 系统将根据 SOAP 报文 Header 中 wsa:from/wsa:Address 和 esb:source/wsa:Address 的内容提取相关系统的服务发起系统和服务源发起系统的系统 ID，并根据该系统 ID 控制系统是否有调用服务的权限，如果都存在调用权限，服务正常受理；否则，一旦有一个系统没有调用权限，则交易会被直接拒绝。

5.3.3 服务调用时间控制

ESB 系统将根据 SOAP 报文中 Header 的 wsu:Created 和 wsu:Expires 进行交易是否超时的判断。ESB 系统会生成当前系统时间，与 Expires 的内容进行比较，如果系统时间比较大则交易超时 ESB 系统进行拒绝；否则交易正常处理。此功能使用的前提条件是该系统与 ESB 系统处于同一个机房，机器存在时钟同步。

5.3.4 报文完整性控制

ESB 系统通过 MAC 进行报文完整性的安全控制。安全控制方案的时序图如下所示：



在报文完整控制过程中使用到了 SOAP 报文 Header 中的 esb:MAC 和 esb:MACOrg 进行控制，并需要调用加密平台的 API 进行处理。ESB 系统会设定是否需要对某一个服务消费者系统进行 MAC 校验。

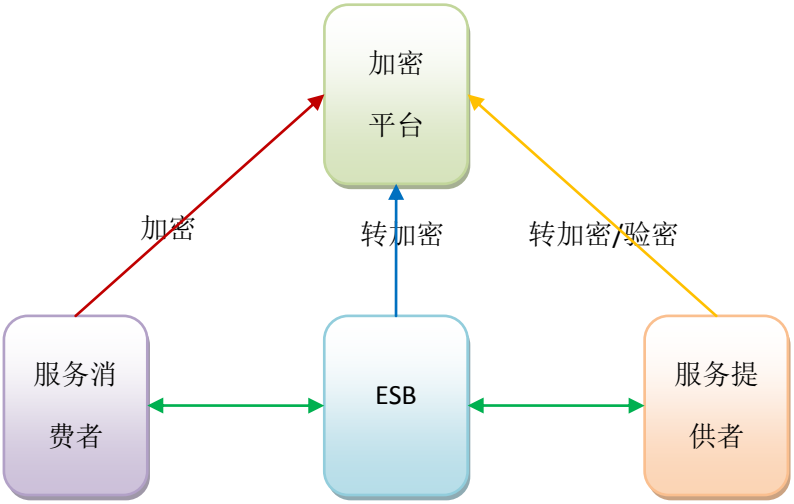
注意：ESB 请求服务提供者系统服务，如果采用标准接出的方式，要求服务提供者系统（新建系统）也实现上述安全方案对 ESB 的请求进行安全控制。采用此种安全控制方法，

ESB 在报文转发的过程中，需要重新生成报文头和进行 MAC 校验码生成。

5.3.5 PIN 字段控制准则

对于银行关键性敏感字段需要加密控制，例如客户登陆密码、查询密码、交易密码、柜员密码或二磁道信息等，通过加密平台进行加密后设置到报文中再进行系统间的数据传输。加密的控制也是采用 SOAP 报文中的 Macorg 字段与加密机进行报文的控制。

ESB 系统在接收到此类字段时，需要进行 PIN 字段的转加密处理。服务消费者、服务提供者、ESB 系统和加密平台的调用关系如下图所示：



一笔含有 PIN 字段，在正常的情况下至少需要访问加密平台 3 次；同时一笔交易的 MAC 校验，也需要访问加密平台 8 次。

5.4 服务流量控制准则

ESB 系统会对服务请求进行流量控制，流量控制包括几个方面的内容：

1) ESB 系统流量控制
2) 服务消费者流量控制
3) 服务的流量控制
4) 服务提供者流量控制

只要有一个维度的并发数超过系统的流量控制，ESB 系统会拒绝该笔服务请求，直接返回流量控制的错误提示，服务提供者不会收到此笔交易请求。

5.5 组合服务控制准则

企业服务总线平台原则上不进行服务的组合，特别是含有逻辑判断的组合服务不能在ESB 平台上实现。

5.6 报文大小限制准则

ESB 将限定报文的大小，SOAP 的报文限制最大报文为 64K 字节。对于 SOP 报文，要求其转换为 SOAP 报文后，报文大小不会超过 64K 的 XML 报文。

5.7 交易超时控制准则

ESB 上的服务请求需要进行超时控制，超时控制模型的准则如下：

1. 服务消费者需要实现超时的控制机制，当超过设定的时间 ESB 仍然没有返回，服务消费者系统可以进入异常流程，交易状态无法确定。该超时时间表示为 C1。
2. ESB 在服务接入容器实现服务的超时机制，超过时间系统没有返回，则直接返回给前端交易超时，交易状态无法确定。前端可能收到超时的异常信息。该时间为 E1。
3. ESB 系统在请求后台服务提供者系统实现服务的超时控制，超过时间后台系统没有返回，则直接返回给前端消费者系统，交易状态无法确定。前端可以收到 ESB 系统与后台超时的异常提示信息。交易状态无法确定。该时间为 E2。
4. 服务提供者系统实现服务的超时控制。该时间为 P1。
5. ESB 系统提供按系统控制超时时间和按服务控制超时时间等两种机制。

一般情况下，这几个超时时间的关系需要满足：

C1 >= E1 > E2 >= P1

5.8 报文分页控制准则

ESB 服务在报文很大的情况下，要求服务提供实现报文的分页查询。分页查询的控制方法可以通过 ESB 报文的应用头进行查询内容的控制。

请求控制字段如下所示：

序号	字段名称	字段名	类型	长度	必需	备注
1.		分页查询结果定位串	String	256	☆	
2.		分页上翻/下翻标识	String	1	☆	
3.		分页每页返回记录数	String	3	☆	

响应控制字段如下所示：

序号	字段名称	字段名	类型	长度	必需	备注
1.		分页上翻/下翻标志			☆	
2.		分页本次返回查询结果记录总数			☆	
3.		分页查询结果的记录总数			☆	
4.		分页定位串数组			☆	
4.1		分页定位串记录			☆	
5.		分页查询结束			☆	

5.9 消息广播控制准则

ESB 系统支持消息的广播控制（发布订阅）。ESB 系统提供存储转发的服务供相关联的系统调用，ESB 收到服务请求后先将消息存储起来，并按照订阅的系统和系统提供的服务接口发送到对应的系统中。

5.10 文件传输控制准则

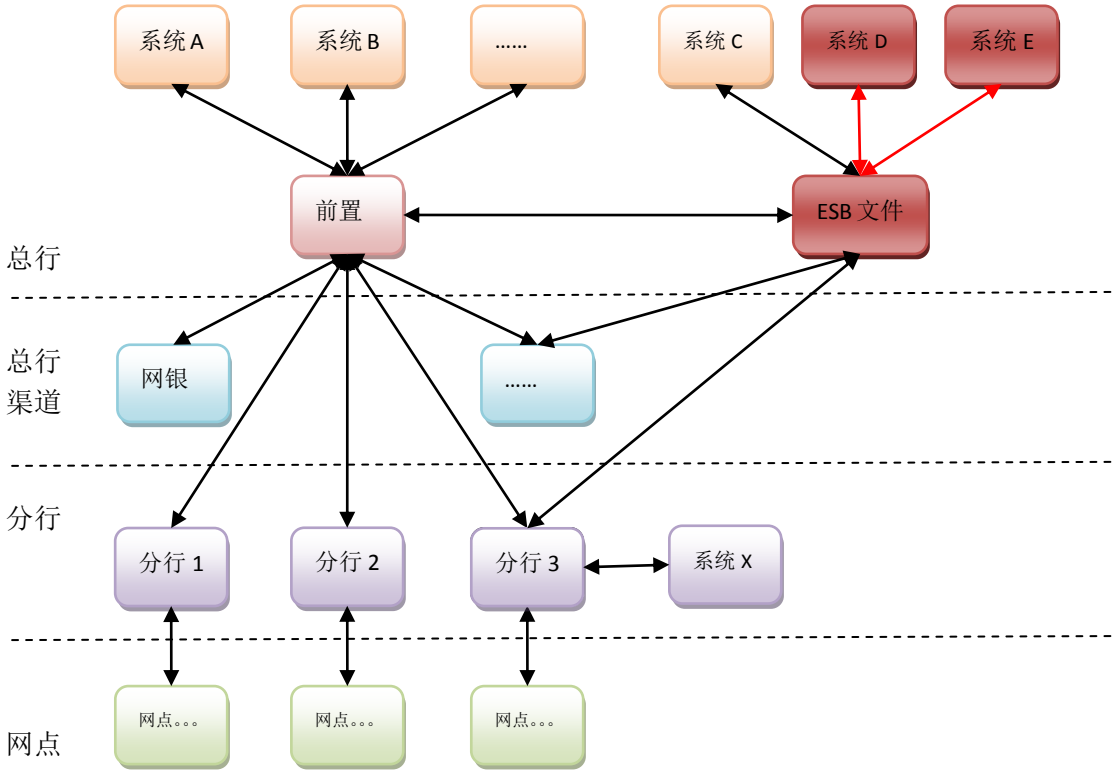
ESB 系统支持文件的传输控制，文件传输的机制兼容浦发银行原有系统的文件传输功能。相关联的系统可以通过两种方式进行文件的传输控制：

1、 使用 ESB 系统文件传输提供的 API 程序进行文件的上传和下载，作为文件传输的客户端。ESB 系统提供 C 语言和 Java 语言的 API 供相关系统调用。

2、 使用 ESB 系统文件传输提供的节点程序部署一个文件服务器的节点，作为文件传输服务群中的一个节点，能实现文件上传、文件下载、文件路由转发等功能。关联系

统可以直接将文件上传到该系统的文件节点或到该文件节点来进行文件的下载。

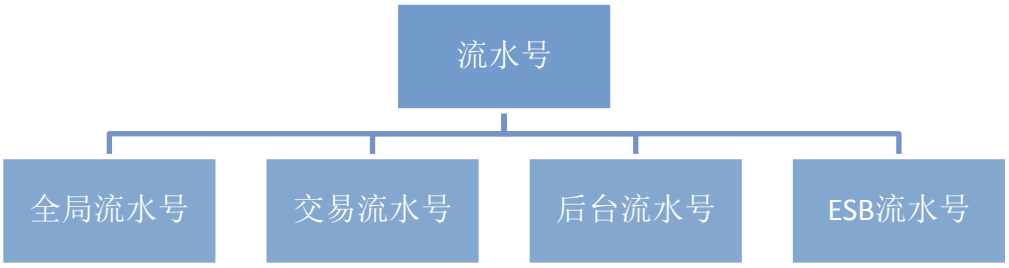
3、新建系统使用 ESB 文件传输提供的程序进行部署，存量系统沿用行内原有的文件程序进行部署。



5.11 流水号控制准则

行内业务系统都有各自的流水号规则，流水号基本都是在两个系统之间传递，能够适应现有的系统架构，但是随着行内系统按照 SOA 进行架构整合，账务处理会分别在各个不同系统中处理，未来的一笔交易可能会涉及多个系统，这种情况下就需要统一流水号在各个系统之间传递准则，并且能够通过流水号来标识和快速跟踪一笔业务在涉及系统的交易状态，所以需要制定统一流水号的规则来适应银行系统架构的变化。

根据流水号在不同情况的使用，可以分为如下几种类型：



每个流水号详细的功能说明如下表所示：

名称	功能描述
全局流水号	唯一标识一笔业务受理的流水号，由交易发起方的系统产生，贯穿整个业务流程，反映一笔业务的唯一标识。例如一笔交易处理过程涉及 2 个系统，出现 2 次服务请求，此 2 次请求的全局流水号不变。
交易流水号	唯一标识一笔前端交易请求的流水号，在两个相邻系统之间传递，由前端系统产生，反映每次调用的唯一标识，后端系统保存前端系统的交易流水号。可以作为冲正的依据。
后台流水号	唯一标识本系统收到一笔交易请求产生的流水号，为服务提供者方产生的流水号，反映后台系统收到一笔交易请求的唯一标识。可以作为冲正的依据，但是容易存在问题。
ESB 流水号	唯一标识 ESB 系统收到一笔交易请求的流水号，技术字段用于系统间差错的定位，不作业务上的控制。相关系统可以选择是否登记到流水中。

主要涉及到全局流水号，交易流水号和系统流水号等，对于新建的系统可以考虑统一编号的规则，下面是一个编号的参考依据：

编码规则：系统 ID（4 位）+交易日期（6 位）+16 为序号

针对不同的业务初始发起方，流水号编码有如下类型：

类型 1：全行集中的发起方，例如：网点平台、网银、客服等。

系统 ID	交易日期	序列号
4 位	6 位：YYMMDD	16 位（每日从 1 排序）

类型 2：全行集中的发起方，由分行业务系统发起，存在多批次、多场次的概念，例如：分行中间业务系统、第三方来报等

系统 ID	交易日期	序列号		
		机构号	批次号	顺序号
4 位	6 位：YYMMDD	6 位	2 位	8 位（每日从 1 排序）

类型 3：全行分散的发起方，无批次概念，例如：ATM 等

系统 ID	交易日期	序列号	
		虚拟柜员号	顺序号

4 位	6 位：YYMMDD	8 位	8 位（每日从 1 排序）

类型 4：全行分散的发起方，有多批次概念，例如：POS 等

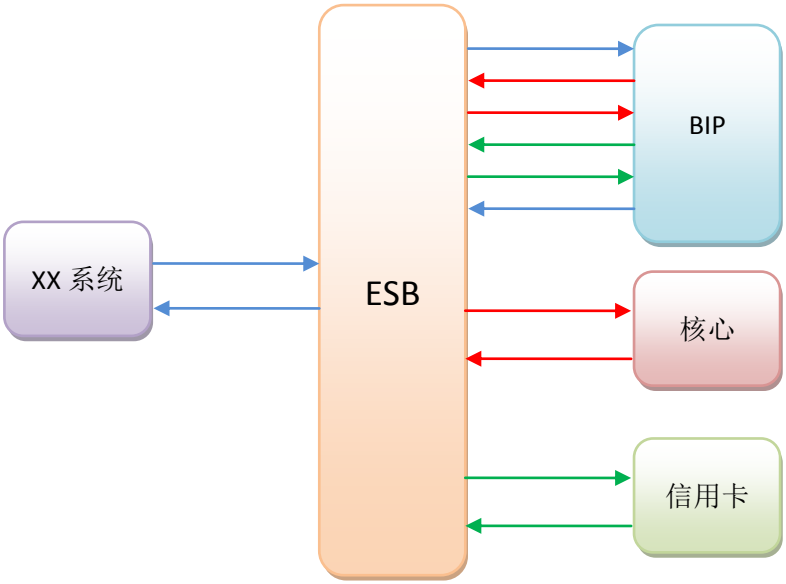
系统 ID	交易日期	序列号		
		虚拟柜员号	批次号	顺序号
4 位	6 位：YYMMDD	8 位	2 位	6 位（每日从 1 排序）

各个字段的详细说明如下所示：

- ✧ 系统 ID：全行标识业务系统的唯一编号，由数据标准组统一编制。
- ✧ 交易日期：交易流程第一个发起系统的账务日期，按 YYMMDD 格式编排。
- ✧ 序列号：16 位，具体编码规则根据系统类型确定，对于一个系统在一天内该序列号不能重复。
- ✧ 批次号：需要按批次对账的系统可以设置批次，不需要按批次对账的系统固定为“00”。
- ✧ 顺序号：由系统生成位的数字序号，流水序号每日从 1 开始计算。
- ✧ 虚拟柜员号：电子渠道系统或电子渠道终端所使用的虚拟柜员号。

在此统一流水号编码的前提下，流水号在系统间传递的控制过程可以通过下列的图表来演示流水产生的过程。交易处理的过程：XX 系统调用 BIP 系统服务，BIP 系统的服务调用核心系统和信用卡系统的其他服务完成业务逻辑的处理。

步骤	流水号产生
1.	XX 系统产生全局流水号 BF 和交易流水号 T1，ESB 产生 ESB 流水号 E1
2.	BIP 收到请求后，产生后台流水号 B1
3.	BIP 请求核心，生成新的交易流水号 T2，引用全局流水号 BF，ESB 产生 ESB 流水号 E2
4.	核心收到请求后，得到 BF 和 T2，并产生后台流水号 B2
5.	BIP 收到核心的响应，得到核心的流水号 B2
6.	BIP 请求信用卡，生成新的流水号 T3，引用全局流水号 BF，ESB 产生 ESB 流水号 E3
7.	信用卡收到请求后，得到 BF 和 T3，产生后台流水号 B3
8.	BIP 收到信用卡的响应，得到信用卡的流水号 B3
9.	XX 系统收到 BIP 的响应，得到流水号 B1，交易结束。
总结	BIP 拥有 BF、T1、T2、B2、T3、B3、B1 的关系，通过 BF 可以在任何系统查询该笔业务的处理情况。XX 系统可以通过 T1 进行交易的冲正，BIP 可以通过 T2 和 T3 进行交易的冲正。



5.12 服务 SLA 控制准则

SLA 的内容和对象的关系如下表所示：

SLA 指标	服务	服务消费者	服务提供者	备注
当前并发用户数				
当前 TPS				
当前业务成功率				
当前系统成功率				
当前平均响应时间				
最大响应时间（超时）				
日交易量				
日系统成功率				
日业务成功率				
日平均响应时间				

5.13 ESB 错误码列表

ESB 系统在系统出现异常的情况下，会提示各类的错误提示信息，错误码会在 SOAP 报文中的 Fault 节点下进行提示。ESB 系统的错误码编码规则如下：

错误码：标识码（2 位）+模块号（2 位）+序号（2 位）

其中，标识码的取值范围如下：

标识码	错误含义
E0	表示请求信息未发送给后端系统，服务消费者系统不需要冲正。
E1	表示请求信息不能确定是否发送给后端系统，服务请求系统需要冲正。
E2	表示请求信息已经发送给后端系统，且后台系统已经响应信息，但是信息不正确，系统处理失败。

而模块号的取值范围如下：

模块号	错误含义
01	安全校验模块
02	服务识别模块
03	服务权限模块
04	有效期控制模块
05	系统隔离模块
06	流量控制模块
07	报文校验模块
08	报文拆包模块
09	报文组包模块
10	系统通讯模块
11	代码转换模块
12	流水记录模块
13	服务组合模块
14	服务路由模块
15	服务前端调度模块
16	服务后端调度模块
17	服务超时控制模块

具体的错误码及处理方式请参照下表进行，后续会根据系统实现补充完整：

错误码	错误提示	处理办法
E00101	IP 地址非法	
E00102	服务权限拒绝	
E00103	MAC 校验失败	
E00404	消息过期	
…待补	…待补	

6 应用举例

6.1 服务消费者服务调用

6.2 服务提供者服务发布

待补。

7 参考附件

7.1 浦发银行信息系统系统编号

7.2 浦发银行 SOP 通讯报文格式

浦发银行SOP最新通讯报文格式2002121